

In []:

Rollno: CO2105 , Name: Sandhya Raghunath
Magadum

Numpy

In [1]:

```
#1
import numpy as np # 'np' is the universal convention
# Two equivalent ways to check the installed version
print(np.__version__)
print(np.version.version)
```

1.26.4

1.26.4

In [2]:

```
#2
import sys
import numpy as np

# A simulated run of 500 environmental sensor logs
lst = list(range(0, 500))
ar = np.arange(500)

print('list taking memory =', sys.getsizeof(lst) * len(lst))
print('numpy array memory =', ar.size * ar.itemsize)
```

list taking memory = 2028000

numpy array memory = 2000

In [3]:

```
#3
# The explicit way to check total data memory buffer size vs wrapper overhead
ar = np.arange(500)
print(ar.nbytes)
print(sys.getsizeof(ar))
```

2000

2112

In [4]:

```
#4
# 0-D array (Scalar sensor flag baseline)
a0 = np.array(101)
print(a0, type(a0), a0.ndim)

# 1-D array (Temperature time-series sequence)
a1 = np.array((22.4, 23.1, 21.8, 24.0, 22.9))
print(a1, type(a1))

# 2-D array from nested tuples (Humidity readings per station zone)
a2 = np.array(((45, 50, 48), (52, 55, 53)))
print(a2)
print(a2.ndim)
```

```
# 2-D array from nested lists
a22 = np.array([[12, 14, 16], [18, 20, 22]])
print(a22.ndim)
```

```
101 <class 'numpy.ndarray'> 0
[22.4 23.1 21.8 24. 22.9] <class 'numpy.ndarray'>
[[45 50 48]
 [52 55 53]]
2
2
```

```
In [5]: #5
# Generating sensor sampling frequencies from 20Hz to 200Hz
a = np.arange(20, 210, 20)
print(a)
print(a.ndim)

[ 20  40  60  80 100 120 140 160 180 200]
1
```

```
In [6]: #6
# Generating exactly 8 calibrated interval points between 0°C and 35°C
a = np.linspace(0, 35, 8)
print(a)

[ 0.  5. 10. 15. 20. 25. 30. 35.]
```

```
In [7]: #7
# 1-D sensor error tracker structures
a = np.zeros(6)
print(a)
a1 = np.ones(4)
print(a1)

# 2-D baseline spatial layout grid matrices
A = np.zeros((3, 4))
B = np.ones((2, 5))
print(A)
print(B)
```

```
[0. 0. 0. 0. 0. 0.]
[1. 1. 1. 1.]
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
```

```
In [8]: #8
# Downcasting baseline float metrics to save memory
A = np.zeros((2, 4))
print(A.astype(np.uint16))
```

```
[[0 0 0 0]
 [0 0 0 0]]
```

```
In [9]: #9
# Generating 6 random humidity percentage values between 30% and 95%
sample = np.random.randint(30, 95, 6)
print(sample)
```

```
[66 93 88 94 61 58]
```

```
In [10]: #10
a = np.array([[15, 25, 35, 45], [55, 65, 75, 85]])
print(a)
print('ndim :', a.ndim)
print('shape:', a.shape)
print('size :', a.size)
print('dtype:', a.dtype)
print('itemsize:', a.itemsize)
print('nbytes:', a.nbytes)
```

```
[[15 25 35 45]
 [55 65 75 85]]
ndim : 2
shape: (2, 4)
size : 8
dtype: int32
itemsize: 4
nbytes: 32
```

```
In [11]: #11
# Verifying trailing comma format on a 1-D sensor reading sequence
b = np.array([12.5, 14.2, 18.9, 15.1])
print(b.shape)
```

```
(4,)
```

```
In [12]: #12
# Reading an air quality metrics scale sequence step-by-step
a = np.arange(15, 100, 8)
print(a)
print(a[2]) # Third sensor node observation value
print(a[-1]) # The most recent trailing pipeline execution check
```

```
[15 23 31 39 47 55 63 71 79 87 95]
31
95
```

```
In [13]: #13
# Indexing inside a 2-D matrix grid of spatial weather tracking cells
mat = np.array([[12, 14, 16], [18, 20, 22]])
print(mat[0, 2]) # Zone 0, station cell 2
print(mat[1, 1]) # Zone 1, station cell 1
```

```
16
20
```

```
In [14]: #14
# Slicing a continuous timeline of laboratory logs
a = np.arange(15, 100, 8)
```

```
print(a)
print(a[1:8:2]) # Extracting elements from index 1 up to 8 (exclusive) skipping eve
```

```
[15 23 31 39 47 55 63 71 79 87 95]
[23 39 55 71]
```

```
In [15]: #15
# Sub-slicing specific blocks inside a 2-D sensor array matrix grid
mat = np.array([[10, 11, 12, 13] , [30, 31, 32, 33]])

print(mat[0:2, 1:3]) # Slice rows 0-1, and columns 1-2
print(mat[:, 0])     # Extract ALL rows, column index 0 only
print(mat[1, :])     # Extract row index 1, ALL columns
```

```
[[11 12]
 [31 32]]
[10 30]
[30 31 32 33]
```

```
In [16]: #16
# Demonstrating why basic slicing creates an uncopied underlying pointer memory VIE
a = np.arange(10)
b = a[2:5]
b[0] = 999
print(a)
```

```
[ 0  1 999  3  4  5  6  7  8  9]
```

```
In [17]: #17
# Pulling discrete, non-sequential target stations out of order via fancy index pos
a = np.arange(15, 100, 8)
print(a)
print(a[[5, 1, 1]]) # Extracts index 5, then index 1 twice
```

```
[15 23 31 39 47 55 63 71 79 87 95]
[55 23 23]
```

```
In [18]: #18
# Isolating sensor data anomalies where values are divisible exactly by 10
a = np.arange(15, 100, 8)
print(a)
mask = (a % 10 == 0)
print(mask)
print(a[mask])
print(a[a % 10 == 0]) # Compact, production-grade vectorized single-line logic
```

```
[15 23 31 39 47 55 63 71 79 87 95]
[False False False False False False False False False False]
[]
[]
```

```
In [19]: #19
# Alternative value run containing divisible values to print an identical trace pat
a = np.arange(10, 80, 7)
print(a)
print(a[a % 2 == 0])
```

```
[10 17 24 31 38 45 52 59 66 73]
[10 24 38 52 66]
```

```
In [20]: #20
# Combining multiple pipeline sensor filtering boundary rules using bitwise operators
a = np.arange(10, 80, 7)
print(a[(a > 20) & (a < 60)]) # AND rule structure
print(a[(a < 20) | (a > 65)]) # OR rule structure
```

```
[24 31 38 45 52 59]
[10 17 66 73]
```

```
In [21]: #21
# Overwriting a specific broken calibration signal matrix value in place
a = np.arange(10, 30, 4)
print(a)
print(a[2])
a[2] = 8888
print(a)
```

```
[10 14 18 22 26]
18
[ 10  14 8888  22  26]
```

```
In [22]: #22
# Appending additional external telemetry logs onto an array tracker
a = np.array([10, 14, 8888, 22, 26])
b = np.append(a, [777, 999])
print(b)
print(a) # Original validation checks confirm structural preservation
```

```
[ 10  14 8888  22  26 777 999]
[ 10  14 8888  22  26]
```

```
In [23]: #23
# Dropping outlier corruption rows using index tracking calls via np.delete()
b = np.array([10, 14, 8888, 22, 26, 777, 999])
c = np.delete(b, 2) # Delete target element at position index 2
print(c)
d = np.delete(c, [1, 3]) # Delete elements currently resting at positions 1 and 3
print(d)
```

```
[ 10  14  22  26 777 999]
[ 10  22 777 999]
```

```
In [24]: #24
# Calibrated environmental index scores out of order
sensor_scores = np.array([54, 18, 92, 23, 76, 12, 65])
sorted_scores = np.sort(sensor_scores)

print(sorted_scores)
print(sensor_scores) # Original copy validation remains unaltered

sensor_scores.sort() # Mutating instance buffer data context in place
print(sensor_scores)
```

```
[12 18 23 54 65 76 92]
[54 18 92 23 76 12 65]
[12 18 23 54 65 76 92]
```

```
In [25]: #25
# Multi-dimensional sorting across axis arguments (Station Readings Matrix)
station_matrix = np.array([[35, 12, 28], [19, 44, 21]])

print(np.sort(station_matrix, axis=1)) # Sorting horizontally across each row
print(np.sort(station_matrix, axis=0)) # Sorting vertically across each column
```

```
[[12 28 35]
 [19 21 44]]
[[19 12 21]
 [35 44 28]]
```

```
In [26]: #26
# Extracting index sort map allocations via argsort()
telemetry_data = np.array([45, 15, 85, 30])
print(np.argsort(telemetry_data))
```

```
[1 3 0 2]
```

```
In [27]: #27
# Tracking explicit isolated modifications
original_log = np.array([120, 140, 160])
copied_log = original_log.copy() # Creating a full standalone deep memory allocation

print(original_log)
print(copied_log)

copied_log[0] = 9999
print('original_log:', original_log)
print('copied_log: ', copied_log)
```

```
[120 140 160]
[120 140 160]
original_log: [120 140 160]
copied_log:   [9999 140 160]
```

```
In [28]: #28
# Bug isolation test tracking split reference views
source_array = np.array([10, 20, 30, 40, 50])
view_subset = source_array[1:4]
view_subset[:] = 0 # Modifying the slice affects the underlying source array
print(source_array)
```

```
[10  0  0  0 50]
```

```
In [29]: #29
# Cleaning up trailing precision decimals from temperature variance metrics
floated_metrics = np.array([14.8923, 22.1056, 19.4567])
print(np.around(floated_metrics, 2))
```

```
[14.89 22.11 19.46]
```

```
In [30]: #30
# 1D array transposition has no visual spatial modification footprint
```

```
one_d_sequence = np.array([5, 10, 15, 20])
print(one_d_sequence.transpose)
print(one_d_sequence.transpose())
print(one_d_sequence.T)
```

<built-in method transpose of numpy.ndarray object at 0x000001FDFCA706F0>

```
[ 5 10 15 20]
[ 5 10 15 20]
```

```
In [31]: #31
# 2D grid conversion tracking matrix transposition properties
zone_grid = np.array([[11, 12, 13], [21, 22, 23]])
print(zone_grid.shape)
print(zone_grid.T)
print(zone_grid.T.shape)
```

```
(2, 3)
[[11 21]
 [12 22]
 [13 23]]
(3, 2)
```

```
In [32]: #32
# Accumulating a full dataset baseline profile block
readings_block = np.array([[10, 15, 20], [25, 30, 35]])
total_sum = np.sum(readings_block)
print(total_sum)
```

```
135
```

```
In [33]: #33
# Evaluation of the full functional family metrics
readings_block = np.array([[10, 15, 20], [25, 30, 35]])
print('sum    :', np.sum(readings_block))
print('mean   :', np.mean(readings_block))
print('std    :', np.std(readings_block))
print('max    :', np.max(readings_block))
print('argmax:', np.argmax(readings_block)) # Flattened index location of maximum v
```

```
sum    : 135
mean   : 22.5
std    : 8.539125638299666
max    : 35
argmax: 5
```

```
In [34]: #34
# A flat 1D sequence of 12 consecutive sensor readings
flat_readings = np.array([10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65])
print("Original shape:", flat_readings.shape)

# Reshaping into a 3x4 grid matrix (3 zones, 4 sensors each)
grid_3x4 = flat_readings.reshape(3, 4)
print("\n3x4 Matrix:\n", grid_3x4)

# Reshaping into a 4x3 grid matrix using the -1 shortcut (automatic calculation)
grid_4x3 = flat_readings.reshape(4, -1)
print("\n4x3 Matrix (using -1):\n", grid_4x3)
```

```
# Flattening a 2D matrix back into a 1D sequence using .reshape(-1) or .flatten()
flattened = grid_3x4.reshape(-1)
print("\nFlattened back to 1D:\n", flattened)
```

Original shape: (12,)

3x4 Matrix:

```
[[10 15 20 25]
 [30 35 40 45]
 [50 55 60 65]]
```

4x3 Matrix (using -1):

```
[[10 15 20]
 [25 30 35]
 [40 45 50]
 [55 60 65]]
```

Flattened back to 1D:

```
[10 15 20 25 30 35 40 45 50 55 60 65]
```

```
In [35]: #35
# Two separate batches of spatial zone sensor data
batch_A = np.array([11, 12, 13, 14])
batch_B = np.array([21, 22, 23, 24])

# 1. Vertical Stacking (vstack) - stacks arrays row-wise to make a 2D matrix
vertical_stack = np.vstack((batch_A, batch_B))
print("Vertical Stack (vstack):\n", vertical_stack)
print("Shape:", vertical_stack.shape)

# 2. Horizontal Stacking (hstack) - appends arrays side-by-side into a Longer 1D ch
horizontal_stack = np.hstack((batch_A, batch_B))
print("\nHorizontal Stack (hstack):\n", horizontal_stack)
print("Shape:", horizontal_stack.shape)
```

Vertical Stack (vstack):

```
[[11 12 13 14]
 [21 22 23 24]]
```

Shape: (2, 4)

Horizontal Stack (hstack):

```
[11 12 13 14 21 22 23 24]
```

Shape: (8,)

```
In [36]: #37
# Baseline temperature and adjustment matrices
temps_morning = np.array([20.5, 22.1, 19.8, 24.2])
temps_evening = np.array([18.2, 21.0, 17.5, 22.0])

# 1. Addition & Subtraction
temp_diff = temps_morning - temps_evening
print("Temperature drop:", temp_diff)

# 2. Multiplication & Division (Calibrating scaling factors)
calibrated_temps = temps_morning * 1.05
print("Calibrated morning temps:", calibrated_temps)
```



```
# 3. Squaring / Exponents
variance_squared = temp_diff ** 2
print("Squared variance:", variance_squared)
```

Temperature drop: [2.3 1.1 2.3 2.2]
 Calibrated morning temps: [21.525 23.205 20.79 25.41]
 Squared variance: [5.29 1.21 5.29 4.84]

```
In [37]: #38
# A 3x3 matrix representing humidity across 3 stations over 3 hours
humidity_matrix = np.array([[42, 47, 50]])

# A 1D array of calibration offsets for the 3 hours (Shape compatible: 1x3 broadcast)
hourly_offset = np.array([2, 5, -1])

# Adding the offset array to every row of the matrix simultaneously
adjusted_humidity = humidity_matrix + hourly_offset
print("Adjusted Humidity Matrix:\n", adjusted_humidity)
```

Adjusted Humidity Matrix:
 [[44 52 49]]

```
In [38]: #39
# Broadcasting a single scalar value across an entire multi-dimensional array
grid_baseline = np.ones((2, 4))
calibrated_grid = grid_baseline * 15.0
print("\nScalar Broadcasting:\n", calibrated_grid)
```

Scalar Broadcasting:
 [[15. 15. 15. 15.]
 [15. 15. 15. 15.]]

```
In [39]: #40
identity = np.eye(3)
print("3x3 Identity Matrix:\n", identity)

# Diagonal Extraction
matrix_data = np.array([[10, 20],
                        [30, 40]])
print("\nExtracted Diagonal elements:", np.diag(matrix_data))
```

3x3 Identity Matrix:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Extracted Diagonal elements: [10 40]

```
In [40]: #41
a = np.arange(20, 210, 20)
print(a)
print(a.ndim)

[ 20  40  60  80 100 120 140 160 180 200]
1
```

```
In [41]: #42
timeline = np.arange(10, 110, 10)
```

```

print("Original timeline:", timeline)

# Splits array at position index 3 and index 7
chunk_1, chunk_2, chunk_3 = np.split(timeline, [3, 7])
print("Chunk 1 (Indices 0-2):", chunk_1)
print("Chunk 2 (Indices 3-6):", chunk_2)
print("Chunk 3 (Indices 7-9):", chunk_3)

```

```

Original timeline: [ 10  20  30  40  50  60  70  80  90 100]
Chunk 1 (Indices 0-2): [10 20 30]
Chunk 2 (Indices 3-6): [40 50 60 70]
Chunk 3 (Indices 7-9): [ 80  90 100]

```

```

In [42]: #43
# A timeline sequence of 9 data samples
samples = np.array([12, 15, 18, 22, 25, 28, 31, 35, 38])

# Split into 3 equal parts
part1, part2, part3 = np.split(samples, 3)
print("Equal Split Part 1:", part1)
print("Equal Split Part 2:", part2)
print("Equal Split Part 3:", part3)

# Split at custom index locations (e.g., cut at index 2 and index 6)
sub1, sub2, sub3 = np.split(samples, [2, 6])
print("\nCustom Index Split 1 (0 to 1):", sub1)
print("Custom Index Split 2 (2 to 5):", sub2)
print("Custom Index Split 3 (6 to end):", sub3)

```

```

Equal Split Part 1: [12 15 18]
Equal Split Part 2: [22 25 28]
Equal Split Part 3: [31 35 38]

```

```

Custom Index Split 1 (0 to 1): [12 15]
Custom Index Split 2 (2 to 5): [18 22 25 28]
Custom Index Split 3 (6 to end): [31 35 38]

```

```

In [43]: #44
a = np.arange(15, 100, 8)
print(a)
print(a[1:8:2])

```

```

[15 23 31 39 47 55 63 71 79 87 95]
[23 39 55 71]

```

```

In [44]: #45
rng = np.random.default_rng(seed=42)
random_ints = rng.integers(low=10, high=50, size=(3, 4))
print("Random Integers:\n", random_ints)
random_floats = rng.random(size=5)
print("\nRandom Floats:\n", random_floats)
normal_dist = rng.normal(loc=20.0, scale=2.5, size=(2, 3))
print("\nNormal Distribution Matrix:\n", normal_dist)
arr = np.array([10, 20, 30, 40, 50])
rng.shuffle(arr) # Mutates arr directly
print("\nShuffled Array:", arr)

```

Random Integers:

```
[[13 40 36 27]
 [27 44 13 37]
 [18 13 31 49]]
```

Random Floats:

```
[0.7611397 0.78606431 0.12811363 0.45038594 0.37079802]
```

Normal Distribution Matrix:

```
[[21.94447984 20.16507674 22.81810302]
 [21.16877336 17.85176884 20.92187696]]
```

Shuffled Array: [20 50 10 30 40]

```
In [45]: #46
# A 2D matrix representing 3 experiment stations tracking 4 test readings each
data_matrix = np.array([
    [15, 25, 35, 45],
    [10, 20, 30, 40],
    [50, 60, 70, 80]
])

# 1. Basic Measures of Central Tendency
print("Global Mean (Average):", np.mean(data_matrix))
print("Global Median:      ", np.median(data_matrix))

# 2. Measures of Dispersion / Spread
print("Standard Deviation:  ", np.std(data_matrix))
print("Variance:           ", np.var(data_matrix))

# 3. Range Analysis
print("Minimum Value:      ", np.min(data_matrix))
print("Maximum Value:      ", np.max(data_matrix))
print("Peak-to-Peak (Max-Min):", np.ptp(data_matrix)) # Peak-to-peak range
```

```
Global Mean (Average): 40.0
Global Median:         37.5
Standard Deviation:    21.01586702153082
Variance:              441.6666666666667
Minimum Value:         10
Maximum Value:         80
Peak-to-Peak (Max-Min): 70
```

```
In [46]: #47
column_means = np.mean(data_matrix, axis=0)
print("Column-wise Means:", column_means)
row_maxes = np.max(data_matrix, axis=1)
print("Row-wise Maxima:  ", row_maxes)
flat_max_idx = np.argmax(data_matrix) # Position index in flattened array
row_max_idx = np.argmax(data_matrix, axis=1) # Position index per row
print("\nFlattened index of max value:", flat_max_idx)
print("Row-by-row index of max value:", row_max_idx)
```

Column-wise Means: [25. 35. 45. 55.]

Row-wise Maxima: [45 40 80]

Flattened index of max value: 11

Row-by-row index of max value: [3 3 3]

In [47]: #48

```
readings = np.array([12, 15, 24, 24, 30, 45, 60, 85])
print("Mean (Average): ", np.mean(readings))
print("Median (Middle Value):", np.median(readings))
print("Variance: ", np.var(readings))
print("Standard Deviation: ", np.std(readings))
print("Minimum Value: ", np.min(readings))
print("Maximum Value: ", np.max(readings))
print("Peak-to-Peak (Range): ", np.ptp(readings))
```

```
Mean (Average):      36.875
Median (Middle Value): 27.0
Variance:            549.109375
Standard Deviation:  23.433082917106745
Minimum Value:      12
Maximum Value:      85
Peak-to-Peak (Range): 73
```

In [48]: #49

```
scores = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
q1 = np.percentile(scores, 25)
median = np.percentile(scores, 50)
q3 = np.percentile(scores, 75)
print("25th Percentile (Q1):", q1)
print("50th Percentile (Q2):", median)
print("75th Percentile (Q3):", q3)
iqr = q3 - q1
print("Interquartile Range: ", iqr)
```

```
25th Percentile (Q1): 32.5
50th Percentile (Q2): 55.0
75th Percentile (Q3): 77.5
Interquartile Range: 45.0
```

In [49]: #50

```
A = np.array([[2, 4], [6, 8]])
B = np.array([[1, 3], [5, 7]])
print("Matrix Addition (A + B):\n", A + B)
print("\nMatrix Subtraction (A - B):\n", A - B)
print("\nElement-wise Multiplication (A * B):\n", A * B)
print("\nElement-wise Division (A / B):\n", A / B)
```

Matrix Addition (A + B):

```
[[ 3  7]
 [11 15]]
```

Matrix Subtraction (A - B):

```
[[1 1]
 [1 1]]
```

Element-wise Multiplication (A * B):

```
[[ 2 12]
 [30 56]]
```

Element-wise Division (A / B):

```
[[2.      1.33333333]
 [1.2     1.14285714]]
```

```
In [50]: #51
identity = np.eye(3)
print("3x3 Identity Matrix:\n", identity)
matrix_data = np.array([[10, 20],
                        [30, 40]])
print("\nExtracted Diagonal elements:", np.diag(matrix_data))
```

3x3 Identity Matrix:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Extracted Diagonal elements: [10 40]

```
In [51]: #52
#The Dynamic Swiss-Army Knife (np.where)
salaries = np.array([4000, -100, 5500, -50])
clean = np.where(salaries < 0, 0, salaries)
print(clean)
```

```
[4000    0 5500    0]
```

```
In [52]: #53
#Instant Outlier Blocker (np.clip)
speeds = np.array([12, 145, -5, 65, 110])
clamped = np.clip(speeds, 0, 100)
print(clamped)
```

```
[ 12 100    0  65 100]
```

```
In [53]: #54
#Quick Matrix Flipping (.T)
grid = np.array([[1, 2, 3],
                 [4, 5, 6]])
print(grid.T)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

```
In [54]: #55
#Fast Boolean Filters
```

```
ages = np.array([14, 25, 19, 32, 17])
adults = ages[ages >= 18]
print(adults)
```

```
[25 19 32]
```

```
In [55]: #56
#Categorical Breakdown (np.unique)
clicks = np.array(['Home', 'Cart', 'Home', 'Home', 'Cart'])
items, counts = np.unique(clicks, return_counts=True)
print(items) # Output: ['Cart' 'Home']
print(counts) # Output: [2 3]
```

```
['Cart' 'Home']
[2 3]
```

```
In [56]: #57
# Quick Shape Shifting (.reshape)
flat_line = np.arange(1, 7) # Creates [1, 2, 3, 4, 5, 6]
matrix = flat_line.reshape(2, -1)
print(matrix)
```

```
[[1 2 3]
 [4 5 6]]
```

```
In [57]: #58
#Index of Maximum Value (np.argmax)
prices = np.array([45, 80, 22, 110, 65])
max_location = np.argmax(prices)
print(max_location)
```

```
3
```

```
In [58]: #59
#Math Ignoring Missing Values (np.nanmean)
scores = np.array([80, 90, np.nan, 70])
average = np.nanmean(scores)
print(average)
```

```
80.0
```

```
In [59]: #60
#Check Condition Thresholds Globally (np.any / np.all)
critical_temps = np.array([22, 19, 35, 24, 21])
print(np.any(critical_temps > 30)) # Output: True
print(np.all(critical_temps < 40)) # Output: True
```

```
True
True
```

```
In [60]: #61
#Flatten Multi-Dimensional Matrix (.flatten)
grid = np.array([[10, 20], [30, 40]])
flat = grid.flatten()
print(flat)
```

```
[10 20 30 40]
```

```
In [61]: #62
#Create Random Mock Test Data (rng.integers)
rng = np.random.default_rng()
mock_data = rng.integers(1, 50, size=(2, 3))
print(mock_data)
```

```
[[14 13 44]
 [41  1 12]]
```

```
In [62]: #63
#Count Item Frequencies (np.bincount)
items = np.array([1, 2, 2, 1, 2, 0])
counts = np.bincount(items) # [1, 2, 3] -> one 0, two 1s, three 2s
print(counts)
```

```
[1 2 3]
```

```
In [63]: #64
#Running Cumulative Sum (np.cumsum)
timeline = np.array([5, 10, 15, 20])
running_total = np.cumsum(timeline)
print(running_total)
```

```
[ 5 15 30 50]
```

```
In [64]: #65
#Matrix Diagonal Extraction (np.diag)
matrix = np.array([[1, 2], [3, 4]])
diagonal = np.diag(matrix)
print(diagonal)
```

```
[1 4]
```

```
In [65]: #66
#Check If Conditions Are False (~ operator)
data = np.array([10, 15, 20, 25])
non_multiples_of_ten = data[~(data % 10 == 0)]
print(non_multiples_of_ten)
```

```
[15 25]
```

```
In [66]: #67
#Fast Search Sorted Position (np.searchsorted)
sorted_bounds = np.array([10, 20, 30, 40])
insert_at = np.searchsorted(sorted_bounds, 25)
print(insert_at)
```

```
2
```

```
In [67]: #68
#Generate Linear Log Steps (np.logspace)
log_steps = np.logspace(1, 3, num=3)
print(log_steps)
```

```
[ 10.  100. 1000.]
```

```
In [68]: #69
#Correlation Matrix (np.corrcoef)
```

```
x = np.array([1, 2, 3, 4])
y = np.array([10, 21, 29, 41])
correlation = np.corrcoef(x, y)
print(correlation[0, 1])
```

0.9973636628446741

```
In [69]: #70
#Isolate Unique Categories (np.unique)
items = np.array(['IT', 'CS', 'IT', 'ETC', 'CS'])
unique_items = np.unique(items)
print(unique_items)
```

['CS' 'ETC' 'IT']

```
In [70]: #71
#Count Unique Occurrences (return_counts)
items = np.array(['IT', 'CS', 'IT', 'ETC', 'CS'])
categories, counts = np.unique(items, return_counts=True)
print(categories) # Output: ['CS' 'ETC' 'IT']
print(counts)    # Output: [2 1 2]
```

['CS' 'ETC' 'IT']
[2 1 2]

```
In [71]: #72
#Get First Occurrence Index Locations (return_index)
items = np.array(['IT', 'CS', 'IT', 'ETC', 'CS'])
categories, first_indices = np.unique(items, return_index=True)
print(categories) # Output: ['CS' 'ETC' 'IT']
print(first_indices) # Output: [1 3 0]
```

['CS' 'ETC' 'IT']
[1 3 0]

```
In [72]: #73
#Reconstruction Map Coordinates (return_inverse)
items = np.array(['IT', 'CS', 'IT', 'ETC', 'CS'])
categories, reconstruction_map = np.unique(items, return_inverse=True)
print(categories) # Output: ['CS' 'ETC' 'IT']
print(reconstruction_map) # Output: [2 0 2 1 0]
```

['CS' 'ETC' 'IT']
[2 0 2 1 0]

```
In [73]: #75
#Check for NaN Elements (np.isnan)
data = np.array([1.5, np.nan, 3.4, np.nan])
missing_mask = np.isnan(data)
print(missing_mask)
```

[False True False True]

In []:

In []:

In []:

Pandas

section 1

```
In [74]: #section 1
import numpy as np
import pandas as pd

# १. वर्जन तपासणे (Checking Versions)
print("Pandas version:", pd.__version__)
print("NumPy version:", np.__version__)
```

Pandas version: 2.3.1

NumPy version: 1.26.4

```
In [75]: #2
# Quick illustration: same data, four containers
data = ['Laptop', 5, 899.99]

py_list = data
py_dict = {'item': 'Laptop', 'count': 5, 'price': 899.99}
np_arr = np.array(data, dtype=object)
pd_series = pd.Series(data, index=['item', 'count', 'price'])

print("List:", py_list)
print("Dict:", py_dict)
print("NumPy array:", np_arr)
print("\nPandas Series (labeled, mixed types):")
print(pd_series)
```

List: ['Laptop', 5, 899.99]

Dict: {'item': 'Laptop', 'count': 5, 'price': 899.99}

NumPy array: ['Laptop' 5 899.99]

Pandas Series (labeled, mixed types):

```
item    Laptop
count      5
price   899.99
dtype: object
```

section 2

```
In [76]: # 3
#Create a Series from a List (Default Integer Index)
lst = ['Rose', 3]
s = pd.Series(lst)

print(s)
print(type(s))
```

```
0    Rose
1         3
dtype: object
<class 'pandas.core.series.Series'>
```

```
In [77]: #4 Create a Series with a Custom Index
lst = ['Rose', 1]
s = pd.Series(lst, index=['s', 'a'])

print(s)
```

```
s    Rose
a         1
dtype: object
```

```
In [78]: #5 Create a Series from a NumPy Array
a = np.array(['apple', 32])
print(a, type(a))

s = pd.Series(a, index=['s', 'a'])

print(s)
```

```
['apple' '32'] <class 'numpy.ndarray'>
s    apple
a         32
dtype: object
```

```
In [79]: #6 Create a Series from a Dictionary
d = {
    'a': [5, 3, 1],
    'b': [1, 3, 5]
}

s = pd.Series(d)

print(s)
```

```
a    [5, 3, 1]
b    [1, 3, 5]
dtype: object
```

```
In [80]: #7 Indexing into a Series
s = pd.Series([70, 80, 90, 100], index=['a', 'b', 'c', 'd'])

print("By label:", s['b'])
print("By position:", s.iloc[1])

print("Slice by label:\n", s['b':'d'])      # Label slicing is inclusive
print("Slice by position:\n", s.iloc[1:3])  # Position slicing is exclusive
```

```

By label: 80
By position: 80
Slice by label:
  b      80
  c      90
  d     100
dtype: int64
Slice by position:
  b      80
  c      90
dtype: int64

```

section 3

```

In [81]: #8 Create DataFrame from a dictionary
s = {'col1': [3, 4]}
df_dict = pd.DataFrame(s)

```

```

print(df_dict)
print(type(df_dict))
print("Shape:", df_dict.shape)

```

```

      col1
0        3
1        4
<class 'pandas.core.frame.DataFrame'>
Shape: (2, 1)

```

```

In [82]: #9 # Create DataFrame from a nested List (must supply column names)
data = [[3, 4, 5], [6, 8, 9]]

df_list = pd.DataFrame(data, columns=['col1', 'col2', 'col3'])
df_list

```

```

Out[82]:
   col1  col2  col3
0      3     4     5
1      6     8     9

```

```

In [83]: # 10 Create DataFrame from a 2D NumPy array
arr = np.array([[1, 2, 3], [5, 3, 2]])

df_arr = pd.DataFrame(arr, columns=['col1', 'col2', 'col3'])
df_arr

```

```

Out[83]:
   col1  col2  col3
0      1     2     3
1      5     3     2

```

```

In [84]: #11 # Create DataFrame from a dictionary of equal-length lists (most common pattern)
df_multi = pd.DataFrame({

```

```

    'name': ['nisha', 'sandhya', 'aarya'],
    'age': [25, 26, 28],
    'score': [88.5, 92.0, 79.3]
})

df_multi

```

Out[84]:

	name	age	score
0	nisha	25	88.5
1	sandhya	26	92.0
2	aarya	28	79.3

section 4

In [85]:

```

# 12
temp_df = pd.DataFrame({'city': ['mumbai', 'delhi', 'bangole', 'hydrabad'], 'temp':
hum_df = pd.DataFrame({'city': ['mumbai', 'delhi', 'hydrabad', 'chennai'], 'humidit

print(temp_df)
print()
print(hum_df)

```

	city	temp
0	mumbai	33
1	delhi	49
2	bangole	85
3	hydrabad	36

	city	humidity
0	mumbai	65
1	delhi	73
2	hydrabad	85
3	chennai	66

In [86]:

```

#13
# concat: stacks rows on top of each other
# (note duplicate index unless ignore_index=True)

pd.concat([temp_df, hum_df], ignore_index=True)

```

Out[86]:

	city	temp	humidity
0	mumbai	33.0	NaN
1	delhi	49.0	NaN
2	bangole	85.0	NaN
3	hydrabad	36.0	NaN
4	mumbai	NaN	65.0
5	delhi	NaN	73.0
6	hydrabad	NaN	85.0
7	chennai	NaN	66.0

In [87]:

```
#14
# concat along columns (axis=1) - aligns by index position

pd.concat([temp_df, hum_df], axis=1)
```

Out[87]:

	city	temp	city	humidity
0	mumbai	33	mumbai	65
1	delhi	49	delhi	73
2	bangole	85	hydrabad	85
3	hydrabad	36	chennai	66

In [88]:

```
#15
# merge: outer join keeps ALL keys from both sides,
# filling NaN where unmatched

pd.merge(temp_df, hum_df, on='city', how='outer')
```

Out[88]:

	city	temp	humidity
0	bangole	85.0	NaN
1	chennai	NaN	66.0
2	delhi	49.0	73.0
3	hydrabad	36.0	85.0
4	mumbai	33.0	65.0

In [89]:

```
# 16
# merge: inner join keeps ONLY keys present in both DataFrames

pd.merge(temp_df, hum_df, on='city', how='inner')
```

Out[89]:

	city	temp	humidity
0	mumbai	33	65
1	delhi	49	73
2	hydrabad	36	85

In [90]: *#17*
merge: left / right joins

```
print(pd.merge(temp_df, hum_df, on='city', how='left'))
print()
print(pd.merge(temp_df, hum_df, on='city', how='right'))
```

	city	temp	humidity
0	mumbai	33	65.0
1	delhi	49	73.0
2	bangole	85	NaN
3	hydrabad	36	85.0

	city	temp	humidity
0	mumbai	33.0	65
1	delhi	49.0	73
2	hydrabad	36.0	85
3	chennai	NaN	66

In [91]: *#18*
join: merges on the INDEX instead of a column

```
left = temp_df.set_index('city')
right = hum_df.set_index('city')

left.join(right, how='outer')
```

Out[91]:

	temp	humidity
city		
bangole	85.0	NaN
chennai	NaN	66.0
delhi	49.0	73.0
hydrabad	36.0	85.0
mumbai	33.0	65.0

section 5

In [92]: *# 19*
#Create a Synthetic Student Dataset
import pandas **as** pd

```

import numpy as np

# Create a synthetic student dataset
np.random.seed(25)
n = 150

df = pd.DataFrame({
    'RollNo': np.arange(1, n + 1),
    'Name': np.random.choice(['Amit', 'Neha', 'Raj', 'Sneha', 'Kiran'], n),
    'Age': np.random.randint(18, 25, n),
    'Marks': np.random.randint(35, 100, n),
    'Attendance': np.round(np.random.uniform(60, 100, n), 1),
    'Department': np.random.choice(['CS', 'IT', 'ENTC', 'Mechanical'], n)
})

# Add missing values and duplicate rows
df.loc[[5, 12, 28], 'Marks'] = np.nan
df = pd.concat([df, df.iloc[[3, 10]]], ignore_index=True)

print(df.shape)
print(df.head())

```

(152, 6)

	RollNo	Name	Age	Marks	Attendance	Department
0	1	Kiran	19	97.0	71.8	IT
1	2	Raj	24	70.0	98.6	CS
2	3	Kiran	22	60.0	66.4	ENTC
3	4	Raj	21	55.0	95.2	IT
4	5	Amit	20	39.0	94.5	ENTC

```

In [93]: #20
#Export Data to CSV, Excel and JSON
# Export files
df.to_csv('student_data.csv', index=False)
df.to_excel('student_data.xlsx', index=False)
df.to_json('student_data.json', orient='records')

print("Files saved successfully.")

```

Files saved successfully.

```

In [94]: #21
#Read Data from CSV, Excel and JSON
# Read files
df_csv = pd.read_csv('student_data.csv')
df_excel = pd.read_excel('student_data.xlsx')
df_json = pd.read_json('student_data.json')

print(df_csv.shape)
print(df_excel.shape)
print(df_json.shape)

print(df_csv.head())

```

```
(152, 6)
```

```
(152, 6)
```

```
(152, 6)
```

	RollNo	Name	Age	Marks	Attendance	Department
0	1	Kiran	19	97.0	71.8	IT
1	2	Raj	24	70.0	98.6	CS
2	3	Kiran	22	60.0	66.4	ENTC
3	4	Raj	21	55.0	95.2	IT
4	5	Amit	20	39.0	94.5	ENTC

section 6

```
In [95]: #22
print(df.shape)
print(df.size)
print(df.ndim)
```

```
(152, 6)
```

```
912
```

```
2
```

```
In [96]: #23
df.dtypes
```

```
Out[96]: RollNo      int32
Name          object
Age           int32
Marks         float64
Attendance     float64
Department     object
dtype: object
```

```
In [97]: #24
print("Column:",list(df.columns))
print("index:",df.index)
```

```
Column: ['RollNo', 'Name', 'Age', 'Marks', 'Attendance', 'Department']
index: RangeIndex(start=0, stop=152, step=1)
```

```
In [98]: #25
df.values[:2]
```

```
Out[98]: array([[1, 'Kiran', 19, 97.0, 71.8, 'IT'],
                [2, 'Raj', 24, 70.0, 98.6, 'CS']], dtype=object)
```

```
In [99]: #26
df.axes
```

```
Out[99]: [RangeIndex(start=0, stop=152, step=1),
          Index(['RollNo', 'Name', 'Age', 'Marks', 'Attendance', 'Department'], dtype='object')]
```

```
In [100... #27
df.keys()
```



```
Out[100... Index(['RollNo', 'Name', 'Age', 'Marks', 'Attendance', 'Department'], dtype='object')
```

```
In [101... #28
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 152 entries, 0 to 151
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   RollNo      152 non-null    int32
1   Name        152 non-null    object
2   Age         152 non-null    int32
3   Marks       149 non-null    float64
4   Attendance  152 non-null    float64
5   Department  152 non-null    object
dtypes: float64(2), int32(2), object(2)
memory usage: 6.1+ KB
```

```
In [102... #29
df.describe()
```

```
Out[102...
      RollNo      Age      Marks      Attendance
count  152.000000  152.000000  149.000000  152.000000
mean    74.605263   21.052632   64.758389   80.336842
std     43.853180    1.914703   18.886092   11.441864
min      1.000000   18.000000   35.000000   60.200000
25%     36.750000   19.750000   47.000000   71.150000
50%     74.500000   21.000000   64.000000   80.400000
75%    112.250000   23.000000   83.000000   90.225000
max    150.000000   24.000000   98.000000   99.400000
```

```
In [103... #30
df.describe(include='object')
```

```
Out[103...
      Name      Department
count      152           152
unique         5           4
top      Amit           CS
freq      35           44
```

section 7

```
In [104... #31  
df.iloc[0]
```

```
Out[104... RollNo      1  
Name      Kiran  
Age       19  
Marks     97.0  
Attendance 71.8  
Department IT  
Name: 0, dtype: object
```

```
In [105... #32  
df.iloc[-1]
```

```
Out[105... RollNo      11  
Name      Sneha  
Age       19  
Marks     44.0  
Attendance 76.9  
Department IT  
Name: 151, dtype: object
```

```
In [106... #33  
df.iloc[[1,6]]
```

```
Out[106...   RollNo  Name  Age  Marks  Attendance  Department  
1         2   Raj   24   70.0         98.6           CS  
6         7  Kiran   22   44.0         63.3           IT
```

```
In [107... #34  
df.iloc[0:3 , 0:4 ]
```

```
Out[107...   RollNo  Name  Age  Marks  
0         1  Kiran   19   97.0  
1         2   Raj   24   70.0  
2         3  Kiran   22   60.0
```

```
In [108... #35  
df.loc[0]
```

```
Out[108... RollNo      1  
Name      Kiran  
Age       19  
Marks     97.0  
Attendance 71.8  
Department IT  
Name: 0, dtype: object
```

In [109...

```
#36  
df.loc[[2,4]]
```

Out[109...

	RollNo	Name	Age	Marks	Attendance	Department
2	3	Kiran	22	60.0	66.4	ENTC
4	5	Amit	20	39.0	94.5	ENTC

In [110...

```
#37  
df.loc[:, 'Age']
```

Out[110...

```
0      19  
1      24  
2      22  
3      21  
4      20  
..  
147    23  
148    20  
149    22  
150    21  
151    19  
Name: Age, Length: 152, dtype: int32
```

In [111...

```
#38  
df.loc[[2,4], 'Name': 'Marks']
```

Out[111...

	Name	Age	Marks
2	Kiran	22	60.0
4	Amit	20	39.0

In [112...

```
#39  
df.loc[:, ['Name', 'Marks']]
```

Out[112...

	Name	Marks
0	Kiran	97.0
1	Raj	70.0
2	Kiran	60.0
3	Raj	55.0
4	Amit	39.0
...	...	...
147	Kiran	87.0
148	Sneha	54.0
149	Neha	85.0
150	Raj	55.0
151	Sneha	44.0

152 rows × 2 columns

In [113...

```
#40
kiran_data = df[df['Name']=='Kiran']
kiran_data.head()
```

Out[113...

	RollNo	Name	Age	Marks	Attendance	Department
0	1	Kiran	19	97.0	71.8	IT
2	3	Kiran	22	60.0	66.4	ENTC
5	6	Kiran	19	NaN	62.1	Mechanical
6	7	Kiran	22	44.0	63.3	IT
8	9	Kiran	20	88.0	75.6	IT

section 8

In [114...

```
#41 Sorting
df.sort_values(by='Marks', ascending = False).head()
```

Out[114...

	RollNo	Name	Age	Marks	Attendance	Department
72	73	Sneha	19	98.0	79.1	IT
0	1	Kiran	19	97.0	71.8	IT
145	146	Amit	19	96.0	93.2	IT
130	131	Neha	24	96.0	68.7	Mechanical
135	136	Neha	21	93.0	90.1	CS

In [115...

```
#42 sort()
df.sort_values(by=['Marks', 'RollNo'], ascending = [True, False]).head()
```

Out[115...

	RollNo	Name	Age	Marks	Attendance	Department
146	147	Raj	20	35.0	69.7	ENTC
113	114	Neha	20	35.0	63.6	CS
68	69	Amit	18	35.0	81.6	Mechanical
31	32	Kiran	21	36.0	74.7	CS
14	15	Sneha	20	36.0	89.3	ENTC

In [116...

```
#43 Rank()
df['Age_rank'] = df['Age'].rank(ascending = False)
df[['Age', 'Age_rank']].head()
```

Out[116...

	Age	Age_rank
0	19	126.0
1	24	11.5
2	22	53.0
3	21	75.0
4	20	100.5

In [117...

```
#44 Rank()
df['Marks_rank'] = df['Marks'].rank(ascending = False)
df[['Marks', 'Marks_rank']].head()
```

Out[117...

	Marks	Marks_rank
0	97.0	2.0
1	70.0	65.5
2	60.0	81.5
3	55.0	89.5
4	39.0	135.0

In [118...

```
#45 rename()
df_renamed = df.rename(columns={'Department':'Dept' , 'Attendance':'Attend'})
df_renamed.head(2)
```

Out[118...

	RollNo	Name	Age	Marks	Attend	Dept	Age_rank	Marks_rank
0	1	Kiran	19	97.0	71.8	IT	126.0	2.0
1	2	Raj	24	70.0	98.6	CS	11.5	65.5

In [119...

```
#46 drop column()
df.drop('Marks' , axis=1 , inplace=True)
df.drop(['Name', 'RollNo'] , axis=1).head(5)
```

Out[119...

	Age	Attendance	Department	Age_rank	Marks_rank
0	19	71.8	IT	126.0	2.0
1	24	98.6	CS	11.5	65.5
2	22	66.4	ENTC	53.0	81.5
3	21	95.2	IT	75.0	89.5
4	20	94.5	ENTC	100.5	135.0

In [120...

```
#47 drop row()
df.drop([0 , 1 , 2 , 3 ] , axis=0).head()
```

Out[120...

	RollNo	Name	Age	Attendance	Department	Age_rank	Marks_rank
4	5	Amit	20	94.5	ENTC	100.5	135.0
5	6	Kiran	19	62.1	Mechanical	126.0	NaN
6	7	Kiran	22	63.3	IT	53.0	123.5
7	8	Neha	23	76.7	CS	32.5	99.0
8	9	Kiran	20	75.6	IT	100.5	20.0

In [121...

```
#48 insert column()
df_copy = df.copy()
```

```
df_copy.insert(1, 'AgeGroup', np.where(df_copy['Age'] >= 80, 'Senior', 'Adult'))
df_copy.head(5)
```

Out[121...

	RollNo	AgeGroup	Name	Age	Attendance	Department	Age_rank	Marks_rank
0	1	Adult	Kiran	19	71.8	IT	126.0	2.0
1	2	Adult	Raj	24	98.6	CS	11.5	65.5
2	3	Adult	Kiran	22	66.4	ENTC	53.0	81.5
3	4	Adult	Raj	21	95.2	IT	75.0	89.5
4	5	Adult	Amit	20	94.5	ENTC	100.5	135.0

section 9

In [122...

```
#49
# map() on a Series with a dictionary
dept_code = {'IT': 1, 'CS': 2, 'ENTC': 3}
df['Dept_code'] = df['Department'].map(dept_code)
df[['Department', 'Dept_code']].head()
```

Out[122...

	Department	Dept_code
0	IT	1.0
1	CS	2.0
2	ENTC	3.0
3	IT	1.0
4	ENTC	3.0

In [123...

```
#50
# map() with a function / lambda
df['Age_in_months'] = df['Age'].map(lambda x: x * 12)
df[['Age', 'Age_in_months']].head()
```

Out[123...

	Age	Age_in_months
0	19	228
1	24	288
2	22	264
3	21	252
4	20	240

In [124...

```
#51
# apply() on a Series - equivalent to map() for simple functions
```

```
df['Age_squared'] = df['Age'].apply(lambda x: x ** 2)
df[['Age', 'Age_squared']].head()
```

Out[124...

	Age	Age_squared
0	19	361
1	24	576
2	22	484
3	21	441
4	20	400

In [125...

```
#52
# apply() on a DataFrame, row-wise (axis=1) - function receives a full row
def student_status(row):
    return 'Good' if (row['Attendance'] > 75 and row['Age'] >= 21) else 'Needs Atte

df['Status'] = df.apply(student_status, axis=1)
df[['Attendance', 'Age', 'Status']].head()
```

Out[125...

	Attendance	Age	Status
0	71.8	19	Needs Attention
1	98.6	24	Good
2	66.4	22	Needs Attention
3	95.2	21	Good
4	94.5	20	Needs Attention

In [126...

```
#53
# apply() on a DataFrame, column-wise (axis=0, the default) - function receives a f
df[['Age', 'Attendance']].apply(lambda col: col.max() - col.min())
```

Out[126...

```
Age          6.0
Attendance    39.2
dtype: float64
```

In [127...

```
#54
# DataFrame.map() - element-wise transform across the WHOLE frame (numeric columns
df[['Age', 'Attendance']].map(lambda x: round(x, 0))
```


Out[127...

	Age	Attendance
0	19	72.0
1	24	99.0
2	22	66.0
3	21	95.0
4	20	94.0
...	...	...
147	23	74.0
148	20	69.0
149	22	98.0
150	21	95.0
151	19	77.0

152 rows × 2 columns

section 10

In [128...

```
#55
# count of missing values per column
df.isnull().sum()
```

Out[128...

```
RollNo      0
Name        0
Age         0
Attendance  0
Department  0
Age_rank    0
Marks_rank  3
Dept_code   35
Age_in_months 0
Age_squared 0
Status      0
dtype: int64
```

In [129...

```
#56
# isna() is an alias for isnull() - identical behavior
df.isna().sum()
```

```
Out[129... RollNo      0
          Name      0
          Age      0
          Attendance 0
          Department 0
          Age_rank   0
          Marks_rank 3
          Dept_code  35
          Age_in_months 0
          Age_squared 0
          Status     0
          dtype: int64
```

```
In [130... #57
# inverse: count of NON-missing values per column
df.notnull().sum()
```

```
Out[130... RollNo      152
          Name      152
          Age      152
          Attendance 152
          Department 152
          Age_rank   152
          Marks_rank 149
          Dept_code  117
          Age_in_months 152
          Age_squared 152
          Status     152
          dtype: int64
```

```
In [131... #58
# dropna(): remove rows with ANY missing value
print("Before:", df.shape)
print("After dropna():", df.dropna().shape)
```

Before: (152, 11)
After dropna(): (117, 11)

```
In [132... #59
# fillna() with a constant
df['Attendance'].fillna(75.0).isnull().sum()
```

```
Out[132... 0
```

```
In [133... #60
# fillna() with the column mean / median (common imputation strategy)
mean_fill = df['Attendance'].fillna(df['Attendance'].mean())
median_fill = df['Attendance'].fillna(df['Attendance'].median())
print("Mean used:", df['Attendance'].mean())
print("Median used:", df['Attendance'].median())
```

Mean used: 80.33684210526316
Median used: 80.4

```
In [134... #61
# fillna() with a GROUPED statistic - more accurate than a single global fill value
```

```
df['Attendance'] = df.groupby('Department')['Attendance'].transform(lambda x: x.fillna(df['Attendance'].isnull().sum()))
```

Out[134...] 0

In [135... #62
interpolate(): fill gaps using linear (or other) interpolation
demo = pd.Series([70.0, np.nan, np.nan, 85.0, 90.0])
print(demo.interpolate())

```
0    70.0
1    75.0
2    80.0
3    85.0
4    90.0
dtype: float64
```

section 11

In [136... #63
total number of duplicate rows (keeps first occurrence as non-duplicate)
df.duplicated().sum()

Out[136...] 2

In [137... #64
duplicates based on a SUBSET of columns
df.duplicated(subset=['Age', 'Department']).sum()

Out[137...] 124

In [138... #65
print("Before:", df.shape)
df_clean = df.drop_duplicates()
print("After drop_duplicates():", df_clean.shape)

```
Before: (152, 11)
After drop_duplicates(): (150, 11)
```

In [139... #66
keep='last' keeps the LAST occurrence instead of the first
df.drop_duplicates(keep='last').shape

Out[139...] (150, 11)

section 12

In [140... #67
df['Department'].str.lower().head()

```
Out[140...] 0      it
            1      cs
            2      entc
            3      it
            4      entc
            Name: Department, dtype: object
```

```
In [141...] #68
# strip() removes leading/trailing whitespace
messy = pd.Series([' IT', 'CS ', ' ENTC '])
messy.str.strip()
```

```
Out[141...] 0      IT
            1      CS
            2      ENTC
            dtype: object
```

```
In [142...] #69
df['Department'].str.replace('ENTC', 'ETC', regex=False).head()
```

```
Out[142...] 0      IT
            1      CS
            2      ETC
            3      IT
            4      ETC
            Name: Department, dtype: object
```

```
In [143...] #70
# split() - break a string column into parts
full_names = pd.Series(['Kiran Rao', 'Raj Mehta', 'Amit Shah'])
full_names.str.split(' ')
```

```
Out[143...] 0      [Kiran, Rao]
            1      [Raj, Mehta]
            2      [Amit, Shah]
            dtype: object
```

```
In [144...] #71
# expand=True returns a DataFrame of split parts
full_names.str.split(' ', expand=True)
```

```
Out[144...]      0      1
0  Kiran    Rao
1  Raj  Mehta
2  Amit   Shah
```

```
In [145...] #72
# contains() - boolean mask for substring matching (supports regex)
df[df['Department'].str.contains('CS')].head(3)
```

Out[145...

	RollNo	Name	Age	Attendance	Department	Age_rank	Marks_rank	Dept_code	Age
1	2	Raj	24	98.6	CS	11.5	65.5	2.0	
7	8	Neha	23	76.7	CS	32.5	99.0	2.0	
13	14	Kiran	21	67.1	CS	75.0	81.5	2.0	



section 13

In [146...

```
#73
# to_datetime() converts strings/objects into proper datetime64 dtype
dates = pd.to_datetime(['2026-01-15', '2026-02-20', '2026-03-05'])
print(dates)
print(dates.dtype)
```

```
DatetimeIndex(['2026-01-15', '2026-02-20', '2026-03-05'], dtype='datetime64[ns]', fr
eq=None)
datetime64[ns]
```

In [147...

```
#74
# Attach a synthetic 'EnrollDate' column to df for demonstration
rng = np.random.default_rng(7)
df['EnrollDate'] = pd.to_datetime('2026-01-01') + pd.to_timedelta(rng.integers(0, 2
df[['Age', 'EnrollDate']].head()
```

Out[147...

	Age	EnrollDate
0	19	2026-07-08
1	24	2026-05-06
2	22	2026-05-17
3	21	2026-06-29
4	20	2026-04-26

In [148...

```
#75
# .dt accessor - extract date components
df['EnrollMonth'] = df['EnrollDate'].dt.month
df['EnrollDay'] = df['EnrollDate'].dt.day_name()
df[['EnrollDate', 'EnrollMonth', 'EnrollDay']].head()
```

Out[148...

	EnrollDate	EnrollMonth	EnrollDay
0	2026-07-08	7	Wednesday
1	2026-05-06	5	Wednesday
2	2026-05-17	5	Sunday
3	2026-06-29	6	Monday
4	2026-04-26	4	Sunday

In [149...

```
#76
# date_range() - generate a sequence of dates
pd.date_range(start='2026-01-01', periods=5, freq='D')
```

Out[149...

```
DatetimeIndex(['2026-01-01', '2026-01-02', '2026-01-03', '2026-01-04',
                '2026-01-05'],
               dtype='datetime64[ns]', freq='D')
```

In [150...

```
#77
# month-end frequency
pd.date_range(start='2026-01-01', periods=4, freq='ME')
```

Out[150...

```
DatetimeIndex(['2026-01-31', '2026-02-28', '2026-03-31', '2026-04-30'], dtype='datetime64[ns]', freq='ME')
```

In [151...

```
#78
# resample() - group time-series data into fixed time buckets (requires a DatetimeIndex)
ts = df.set_index('EnrollDate').sort_index()
monthly_enrollments = ts['Age'].resample('ME').count()
monthly_enrollments
```

Out[151...

```
EnrollDate
2026-01-31    24
2026-02-28    16
2026-03-31    23
2026-04-30    27
2026-05-31    24
2026-06-30    23
2026-07-31    15
Freq: ME, Name: Age, dtype: int64
```

section 14

In [152...

```
#79
# Split + apply (mean) + combine, in one line
df.groupby('Department')['Age'].mean()
```

```
Out[152...] Department
CS          21.590909
ENTC        21.666667
IT           20.575000
Mechanical  20.342857
Name: Age, dtype: float64
```

```
In [153...] #80
# size() - count of rows per group
df.groupby('Department').size()
```

```
Out[153...] Department
CS          44
ENTC        33
IT           40
Mechanical  35
dtype: int64
```

```
In [154...] #81
# get_group() pull out a single group as its own DataFrame
df.groupby('Department').get_group('IT').head()
```

```
Out[154...]   RollNo  Name  Age  Attendance  Department  Age_rank  Marks_rank  Dept_code  Age_
```

0	1	Kiran	19	71.8	IT	126.0	2.0	1.0
3	4	Raj	21	95.2	IT	75.0	89.5	1.0
6	7	Kiran	22	63.3	IT	53.0	123.5	1.0
8	9	Kiran	20	75.6	IT	100.5	20.0	1.0
9	10	Neha	22	71.4	IT	53.0	119.5	1.0



```
In [155...] #82
# agg() - apply multiple aggregation functions at once
df.groupby('Department')['Age'].agg(['mean', 'min', 'max', 'count'])
```

```
Out[155...]   mean  min  max  count
```

Department				
CS	21.590909	18	24	44
ENTC	21.666667	18	24	33
IT	20.575000	18	24	40
Mechanical	20.342857	18	24	35

```
In [156... #83
# agg() with different functions per column
df.groupby('Department').agg({
    'Age': 'mean',
    'Attendance': ['mean', 'max'],
    'Marks_rank': 'sum'
})
```

```
Out[156...
           Age      Attendance  Marks_rank
           mean      mean  max           sum
Department
CS      21.590909  80.343182   99.3      3090.5
ENTC    21.666667  83.087879   99.4      2559.5
IT      20.575000  78.765000   98.7      3097.5
Mechanical 20.342857  79.531429   98.7      2427.5
```

```
In [157... #84
# transform() - returns an output the SAME SHAPE as the input
df['Dept_age_mean'] = df.groupby('Department')['Age'].transform('mean')
df[['Department', 'Age', 'Dept_age_mean']].head()
```

```
Out[157...
   Department  Age  Dept_age_mean
0          IT   19      20.575000
1          CS   24      21.590909
2         ENTC   22      21.666667
3          IT   21      20.575000
4         ENTC   20      21.666667
```

```
In [158... #85
# filter() - keep only groups that satisfy a condition
df.groupby('Department').filter(lambda g: g['Age'].mean() > 20).head()
```


Out[158...

	RollNo	Name	Age	Attendance	Department	Age_rank	Marks_rank	Dept_code	Age_
0	1	Kiran	19	71.8	IT	126.0	2.0	1.0	
1	2	Raj	24	98.6	CS	11.5	65.5	2.0	
2	3	Kiran	22	66.4	ENTC	53.0	81.5	3.0	
3	4	Raj	21	95.2	IT	75.0	89.5	1.0	
4	5	Amit	20	94.5	ENTC	100.5	135.0	3.0	

section 15

In [159...

```
#86
print("sum:", df['Age'].sum())
print("mean:", df['Age'].mean())
print("median:", df['Age'].median())
print("mode:\n", df['Department'].mode())
```

```
sum: 3200
mean: 21.05263157894737
median: 21.0
mode:
0    CS
Name: Department, dtype: object
```

In [160...

```
#87
print("std:", df['Age'].std())
print("var:", df['Age'].var())
print("count:", df['Age'].count())
print("nunique:", df['Department'].nunique())
```

```
std: 1.9147025210621438
var: 3.666085744161729
count: 152
nunique: 4
```

In [161...

```
#88
print("min:", df['Age'].min())
print("max:", df['Age'].max())
```

```
min: 18
max: 24
```

In [162...

```
#89
df['Attendance'].nlargest()
```

```
Out[162...] 125    99.4
            112    99.3
            129    99.2
            25    98.9
            49    98.8
            Name: Attendance, dtype: float64
```

```
In [163...] #90
df['Attendance'].nsmallest(3)
```

```
Out[163...] 78    60.2
            143    60.5
            75    60.9
            Name: Attendance, dtype: float64
```

```
In [164...] #91
df['Department'].value_counts()
```

```
Out[164...] Department
CS                44
IT                40
Mechanical        35
ENTC              33
            Name: count, dtype: int64
```

```
In [165...] #92
df['Department'].value_counts(normalize=True)
```

```
Out[165...] Department
CS                0.289474
IT                0.263158
Mechanical        0.230263
ENTC              0.217105
            Name: proportion, dtype: float64
```

```
In [166...] #93
# Combine several aggregations into a single summary table
df.select_dtypes(include=np.number).agg(['mean', 'std', 'min', 'max']).T
```

Out[166...

	mean	std	min	max
RollNo	74.605263	43.853180	1.000000	150.000000
Age	21.052632	1.914703	18.000000	24.000000
Attendance	80.336842	11.441864	60.200000	99.400000
Age_rank	76.500000	43.534754	11.500000	145.000000
Marks_rank	75.000000	43.141189	1.000000	148.000000
Dept_code	1.940171	0.791012	1.000000	3.000000
Age_in_months	252.631579	22.976430	216.000000	288.000000
Age_squared	446.855263	80.932104	324.000000	576.000000
EnrollMonth	3.921053	1.913792	1.000000	7.000000
Dept_age_mean	21.052632	0.586541	20.342857	21.666667

In []:

50 Coding Practice Quetions (Easy to Hard)

1 to 15 Easy

In [167...

```
# 1. Create a 1-D array of numbers 1 to 20.
arr1 = np.arange(1, 21)
print(arr1)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

In [168...

```
# 2. Create a 5x5 array of all zeros.
arr2 = np.zeros((5, 5))
print(arr2)
```

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

In [169...

```
# 3. Create a 4x4 identity matrix.
arr3 = np.eye(4)
print(arr3)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

In [170...

```
# 4. Create an array of 10 evenly spaced numbers between 0 and 1.
arr4 = np.linspace(0, 1, 10)
```

```
print(arr4)
```

```
[0.          0.11111111 0.22222222 0.33333333 0.44444444 0.55555556
 0.66666667 0.77777778 0.88888889 1.          ]
```

```
In [171...] # 5. Reverse a given 1-D array.
given_arr5 = np.array([10, 20, 30, 40, 50])
reversed_arr5 = given_arr5[::-1]
print(reversed_arr5)
```

```
[50 40 30 20 10]
```

```
In [172...] # 6. Find the shape, size, and dtype of a given array.
given_arr6 = np.array([[1, 2, 3], [4, 5, 6]])
shape = given_arr6.shape
size = given_arr6.size
dtype = given_arr6.dtype
print(f"Shape: {shape}, Size: {size}, Dtype: {dtype}")
```

```
Shape: (2, 3), Size: 6, Dtype: int32
```

```
In [173...] # 7. Replace all odd numbers in an array with -1.
given_arr7 = np.array([1, 2, 3, 4, 5, 6])
given_arr7[given_arr7 % 2 != 0] = -1
print(given_arr7)
```

```
[-1  2 -1  4 -1  6]
```

```
In [174...] # 8. Find the sum, mean, and max of an array.
given_arr8 = np.array([10, 20, 30, 40])
arr_sum = given_arr8.sum()
arr_mean = given_arr8.mean()
arr_max = given_arr8.max()
print(f"Sum: {arr_sum}, Mean: {arr_mean}, Max: {arr_max}")
```

```
Sum: 100, Mean: 25.0, Max: 40
```

```
In [175...] # 9. Create a 3x3 array of random integers between 1 and 100.
arr9 = np.random.randint(1, 101, size=(3, 3))
print(arr9)
```

```
[[34 49 24]
 [20  7 30]
 [30 64 45]]
```

```
In [176...] # 10. Stack two 1-D arrays vertically and horizontally.
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
v_stacked = np.vstack((a, b))
h_stacked = np.hstack((a, b))
print("Vertical:\n", v_stacked)
print("Horizontal:\n", h_stacked)
```

```
Vertical:
[[1 2 3]
 [4 5 6]]
Horizontal:
[1 2 3 4 5 6]
```

```
In [177... # 11. Extract every 3rd element from an array.
given_arr11 = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])
every_3rd = given_arr11[::3]
print(every_3rd)
```

```
[10 40 70]
```

```
In [178... # 12. Find common elements between two arrays.
arr12_a = np.array([1, 2, 3, 4])
arr12_b = np.array([3, 4, 5, 6])
common = np.intersect1d(arr12_a, arr12_b)
print(common)
```

```
[3 4]
```

```
In [179... # 13. Convert a list of temperatures in Celsius to Fahrenheit using vectorized math
celsius = np.array([0, 20, 37, 100])
fahrenheit = (celsius * 9/5) + 32
print(fahrenheit)
```

```
[ 32.   68.   98.6 212. ]
```

```
In [180... # 14. Count the number of even numbers in an array.
given_arr14 = np.array([1, 2, 3, 4, 5, 6, 7, 8])
even_count = np.sum(given_arr14 % 2 == 0)
print(even_count)
```

```
4
```

```
In [181... # 15. Create an array and compute its transpose.
arr15 = np.array([[1, 2], [3, 4]])
transpose_arr15 = arr15.T
print("Original:\n", arr15)
print("Transpose:\n", transpose_arr15)
```

```
Original:
```

```
[[1 2]
 [3 4]]
```

```
Transpose:
```

```
[[1 3]
 [2 4]]
```

```
In [ ]:
```

15 to 35 (Medium)

```
In [182... # 16. Normalize an array to the range [0, 1] using broadcasting.
arr16 = np.array([10, 20, 30, 40, 50])
normalized = (arr16 - arr16.min()) / (arr16.max() - arr16.min())
print(normalized)
```

```
[0.   0.25 0.5  0.75 1. ]
```

```
In [183... # 17. Find the indices of all elements greater than the array's mean.
arr17 = np.array([1, 5, 2, 9, 3, 8])
```

```
indices = np.where(arr17 > arr17.mean())
print("Indices:", indices[0])
```

Indices: [1 3 5]

In [184... *# 18. Create a checkerboard pattern (8x8) of 0s and 1s using slicing.*

```
checkerboard = np.zeros((8, 8), dtype=int)
checkerboard[1::2, ::2] = 1
checkerboard[:, 1::2] = 1
print(checkerboard)
```

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

In [185... *# 19. Compute the row-wise and column-wise sum of a 2-D matrix.*

```
matrix19 = np.array([[1, 2, 3], [4, 5, 6]])
row_sum = matrix19.sum(axis=1)
col_sum = matrix19.sum(axis=0)
print("Row sums:", row_sum)
print("Column sums:", col_sum)
```

Row sums: [6 15]
Column sums: [5 7 9]

In [186... *# 20. Replace all NaN values in an array with the column mean.*

```
matrix20 = np.array([[1.0, 2.0, np.nan], [4.0, np.nan, 6.0], [7.0, 8.0, 9.0]])
col_means = np.nanmean(matrix20, axis=0)
inds = np.where(np.isnan(matrix20))

# Correctly map the column mean to the missing NaN indices
matrix20[inds] = col_means[inds[1]]
print(matrix20)
```

```
[[1.  2.  7.5]
 [4.  5.  6. ]
 [7.  8.  9. ]]
```

In [187... *# 21. Find the second-largest value in an array without sorting the whole array.*

```
arr21 = np.array([3, 1, 4, 4, 5, 5, 2])
unique_arr = np.unique(arr21)
second_largest = np.partition(unique_arr, -2)[-2]
print("Second largest unique value:", second_largest)
```

Second largest unique value: 4

In [188... *# 22. Given two matrices, check if they can be multiplied, then multiply them.*

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6, 7], [8, 9, 10]])

# Inner dimensions must match for matrix multiplication: columns of A == rows of B
if A.shape[1] == B.shape[0]:
    result = np.dot(A, B)
```

```

    print("Multiplication successful:\n", result)
else:
    print("Matrices cannot be multiplied due to incompatible shapes.")

```

Multiplication successful:
[[21 24 27]
[47 54 61]]

In [189... *# 23. Implement min-max scaling on a feature matrix (axis=0).*

```

X = np.array([[1, 200], [2, 400], [3, 100]])
X_min = X.min(axis=0)
X_max = X.max(axis=0)
scaled_X = (X - X_min) / (X_max - X_min)
print(scaled_X)

```

```

[[0.         0.33333333]
 [0.5        1.         ]
 [1.         0.         ]]

```

In [190... *# 24. Generate a random matrix and compute its inverse and determinant.*

```

matrix24 = np.random.rand(3, 3)
det = np.linalg.det(matrix24)

if det != 0:
    inv = np.linalg.inv(matrix24)
    print("Inverse:\n", inv)
    print("Determinant:", det)

```

Inverse:
[[1.209814 -0.91806719 0.06095575]
[-6.97554104 1.14880855 7.82117258]
[3.03214308 0.73481858 -3.49501062]]
Determinant: -0.08528745665980565

In [191... *# 25. Given an array of exam scores, bucket them into grade categories using boolean indexing.*

```

scores = np.array([95, 82, 67, 75, 89])
grades = np.empty(scores.shape, dtype=object)

grades[scores >= 90] = 'A'
grades[(scores >= 80) & (scores < 90)] = 'B'
grades[(scores >= 70) & (scores < 80)] = 'C'
grades[scores < 70] = 'F'
print(grades)

```

```
['A' 'B' 'F' 'C' 'B']
```

In [192... *# 26. Flatten a 3-D array and confirm whether the result is a view or copy.*

```

arr26 = np.arange(8).reshape(2, 2, 2)
flattened = arr26.ravel()
is_view = flattened.base is arr26
print(f"Is view? {is_view}")

```

Is view? False

In [193... *# 27. Compute the moving average of a 1-D array with a window size of 3.*

```

arr27 = np.array([1, 2, 3, 4, 5, 6])
moving_avg = np.convolve(arr27, np.ones(3)/3, mode='valid')
print(moving_avg)

```

```
[2. 3. 4. 5.]
```

```
In [194... # 28. Implement one-hot encoding for an array of class labels using NumPy only.
labels = np.array([0, 2, 1, 0, 2])
num_classes = np.max(labels) + 1
one_hot = np.eye(num_classes)[labels]
print(one_hot)
```

```
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]
```

```
In [195... # 29. Given a confusion matrix, compute accuracy using only NumPy operations.
cm = np.array([[50, 2, 1], [3, 45, 2], [1, 0, 48]])
accuracy = np.trace(cm) / np.sum(cm)
print("Accuracy:", accuracy)
```

Accuracy: 0.9407894736842105

```
In [196... # 30. Shuffle a feature matrix x and label vector y together using the same permutation
x = np.array([[1, 2], [3, 4], [5, 6]])
y = np.array([0, 1, 0])

permutation = np.random.permutation(len(x))
x_shuffled = x[permutation]
y_shuffled = y[permutation]
print("Shuffled X:\n", x_shuffled)
print("Shuffled Y:", y_shuffled)
```

Shuffled X:

```
[[1 2]
 [3 4]
 [5 6]]
```

Shuffled Y: [0 1 0]

```
In [197... # 31. Compute pairwise Euclidean distances between two sets of points (no loops).
P1 = np.array([[0, 0], [1, 1]])
P2 = np.array([[1, 0], [0, 1], [2, 2]])
dists = np.sqrt(np.sum(P1**2, axis=1, keepdims=True) + np.sum(P2**2, axis=1) - 2 *
print(dists)
```

```
[[1.         1.         2.82842712]
 [1.         1.         1.41421356]]
```

```
In [198... # 32. Implement the sigmoid and ReLU functions and apply them to a random array.
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def relu(x):
    return np.maximum(0, x)
arr32 = np.array([-2, -1, 0, 1, 2])
print("Sigmoid:", sigmoid(arr32))
print("ReLU:", relu(arr32))
```

Sigmoid: [0.11920292 0.26894142 0.5 0.73105858 0.88079708]
ReLU: [0 0 0 1 2]


```
In [199... # 33. Split a dataset array into 80/20 train/test using slicing after shuffling.
dataset = np.arange(10).reshape(5, 2)
np.random.shuffle(dataset)
split_idx = int(0.8 * len(dataset))
train, test = dataset[:split_idx], dataset[split_idx:]
print("Train:\n", train)
print("Test:\n", test)
```

```
Train:
[[0 1]
 [8 9]
 [2 3]
 [4 5]]
Test:
[[6 7]]
```

```
In [200... # 34. Given an array of RGB pixel values, convert it to grayscale using a weighted
rgb_image = np.random.randint(0, 256, (4, 4, 3))
weights = np.array([0.2989, 0.5870, 0.1140])
grayscale = np.dot(rgb_image, weights)
print(grayscale)
```

```
[[174.2203 225.3194 147.4935 145.3048]
 [ 81.9999  96.4967 239.2672 103.709 ]
 [159.8609 150.8037 187.1194  73.3008]
 [166.4569  42.0968 216.6966  94.1245]]
```

```
In [201... # 35. Find duplicate rows in a 2-D array.
matrix35 = np.array([[1, 2], [3, 4], [1, 2], [5, 6]])
unique_rows, counts = np.unique(matrix35, axis=0, return_counts=True)
duplicate_rows = unique_rows[counts > 1]
print("Duplicate rows:\n", duplicate_rows)
```

```
Duplicate rows:
[[1 2]]
```

```
In [ ]:
```

36 to 50 Hard

```
In [202... # 36. Implement a simple linear regression using the normal equation (no Scikit-Lea
# Theta = (X^T * X)^(-1) * X^T * y
X = np.array([[1, 2], [1, 3], [1, 4], [1, 5]]) # Includes bias column
y = np.array([5, 7, 9, 11])
theta = np.linalg.inv(X.T @ X) @ X.T @ y
print("Weights:", theta)
```

```
Weights: [1. 2.]
```

```
In [203... # 37. Implement K-Means cluster assignment step (assign each point to its nearest c
points = np.array([[1, 2], [5, 8], [1, 3], [6, 7]]) # Shape: (N, D)
centroids = np.array([[1, 1], [5, 5]]) # Shape: (K, D)

# Distances shape: (N, K) via broadcasting (N, 1, D) - (K, D)
distances = np.linalg.norm(points[:, np.newaxis, :] - centroids, axis=2)
```

```
assignments = np.argmin(distances, axis=1)
print("Cluster Assignments:", assignments)
```

Cluster Assignments: [0 1 0 1]

```
In [204... # 38. Implement PCA from scratch using np.linalg.eig on the covariance matrix.
X = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0]])
X_centered = X - np.mean(X, axis=0)

cov_matrix = np.cov(X_centered, rowvar=False)
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Sort components by highest variance
idx = np.argsort(eigenvalues)[::-1]
components = eigenvectors[:, idx]
X_projected = X_centered @ components
print("Projected Data:\n", X_projected)
```

Projected Data:

```
[[ 0.44362444 -0.20099093]
 [-2.17719404 -0.05500992]
 [ 0.57071239  0.36808609]
 [-0.12902465  0.06747325]
 [ 1.29188186 -0.17955849]]
```

```
In [205... # 39. Implement a single forward pass of a 2-layer neural network using only NumPy
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

X = np.array([[0.5, 0.1]]) # Batch size 1, 2 features
W1 = np.random.randn(2, 3) # Layer 1 weights
W2 = np.random.randn(3, 1) # Layer 2 weights

hidden = sigmoid(X @ W1)
output = sigmoid(hidden @ W2)
print("Network Output:", output)
```

Network Output: [[0.40891358]]

```
In [206... # 40. Implement softmax (numerically stable version, subtracting the max) for a bat
logits = np.array([[1.0, 2.0, 3.0], [1000, 1001, 1002]])
max_logits = np.max(logits, axis=1, keepdims=True)

exp_logits = np.exp(logits - max_logits)
softmax_probs = exp_logits / np.sum(exp_logits, axis=1, keepdims=True)
print("Softmax Probabilities:\n", softmax_probs)
```

Softmax Probabilities:

```
[[0.09003057 0.24472847 0.66524096]
 [0.09003057 0.24472847 0.66524096]]
```

```
In [207... # 41. Implement gradient descent to fit a line y = mx + b to noisy data.
x = np.array([1, 2, 3, 4, 5])
y = np.array([2.1, 3.9, 6.1, 8.0, 10.2])

m, b = 0.0, 0.0
lr = 0.01
epochs = 1000
```

```

n = len(x)

for _ in range(epochs):
    y_pred = m * x + b
    dm = (-2/n) * np.sum(x * (y - y_pred))
    db = (-2/n) * np.sum(y - y_pred)
    m -= lr * dm
    b -= lr * db

print(f"Fitted line: y = {m:.2f}x + {b:.2f}")

```

Fitted line: y = 2.02x + -0.01

In [208...

```

# 42. Compute a sliding-window (convolution-style) sum over a 1-D signal without us
signal = np.array([1, 2, 3, 4, 5, 6])
window_size = 3

# Use stride tricks to build windows cleanly
shape = (len(signal) - window_size + 1, window_size)
strides = (signal.strides[0], signal.strides[0])
windows = np.lib.stride_tricks.as_strided(signal, shape=shape, strides=strides)

window_sums = np.sum(windows, axis=1)
print("Sliding Window Sums:", window_sums)

```

Sliding Window Sums: [6 9 12 15]

In [209...

```

# 43. Implement batch normalization (subtract mean, divide by std, per feature) for
X = np.array([[1.0, 20.0], [2.0, 30.0], [3.0, 40.0]]) # Shape: (Batch, Features)
eps = 1e-5

mean = np.mean(X, axis=0)
var = np.var(X, axis=0)
X_norm = (X - mean) / np.sqrt(var + eps)
print("Normalized Batch:\n", X_norm)

```

Normalized Batch:

```

[[-1.22473569 -1.22474478]
 [ 0.          0.         ]
 [ 1.22473569  1.22474478]]

```

In [210...

```

# 44. Given a dataset with outliers, implement an IQR-based outlier removal function
data = np.array([10, 12, 12, 13, 12, 11, 100, 13, 12, 11, -50])

q25, q75 = np.percentile(data, [25, 75])
iqr = q75 - q25
lower_bound = q25 - 1.5 * iqr
upper_bound = q75 + 1.5 * iqr

clean_data = data[(data >= lower_bound) & (data <= upper_bound)]
print("Cleaned Data:", clean_data)

```

Cleaned Data: [10 12 12 13 12 11 13 12 11]

In [211...

```

# 45. Implement nearest-neighbor classification (1-NN) using only NumPy distance co
X_train = np.array([[1, 1], [2, 2], [5, 5], [6, 6]])
y_train = np.array(['A', 'A', 'B', 'B'])
X_test = np.array([1.8, 1.9], [5.2, 6.0])

```

```
# Broad-casted absolute distance Logic
dists = np.linalg.norm(X_test[:, np.newaxis, :] - X_train, axis=2)
nearest_indices = np.argmin(dists, axis=1)
predictions = y_train[nearest_indices]
print("Predictions:", predictions)
```

Predictions: ['A' 'B']

```
In [212... # 46. Build a simple image-patch extractor: given an image array and patch size, ex
image = np.arange(16).reshape(4, 4) # 4x4 image matrix
p_h, p_w = 2, 2 # Patch dimensions

# Native reshape/transpose tricks to split spatial planes without copies
h, w = image.shape
patches = image.reshape(h // p_h, p_h, w // p_w, p_w).swapaxes(1, 2).reshape(-1, p_
print("Extracted Patches:\n", patches)
```

Extracted Patches:

```
[[[ 0  1]
  [ 4  5]]
```

```
[[ 2  3]
 [ 6  7]]
```

```
[[ 8  9]
 [12 13]]
```

```
[[10 11]
 [14 15]]]
```

```
In [213... # 47. Compute cosine similarity between two sets of vectors (matrix form, no loops)
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8], [1, 0]])

dot_product = A @ B.T
norm_A = np.linalg.norm(A, axis=1, keepdims=True)
norm_B = np.linalg.norm(B, axis=1, keepdims=True)

cosine_sim = dot_product / (norm_A @ norm_B.T)
print("Cosine Similarity Matrix:\n", cosine_sim)
```

Cosine Similarity Matrix:

```
[[0.97341717 0.96761727 0.4472136 ]
 [0.99868766 0.99716412 0.6       ]]
```

```
In [214... # 48. Implement a basic Markov chain transition matrix and compute the stationary d
P = np.array([[0.7, 0.3], [0.4, 0.6]]) # Row-stochastic transition matrix

# Find Left eigenvectors by computing right eigenvectors of P.T
eigenvalues, eigenvectors = np.linalg.eig(P.T)
stationary_vector = eigenvectors[:, np.isclose(eigenvalues, 1.0)].real.flatten()

# Normalize vector so probabilities equal 1
stationary_distribution = stationary_vector / np.sum(stationary_vector)
print("Stationary Distribution:", stationary_distribution)
```

Stationary Distribution: [0.57142857 0.42857143]

```
In [215... # 49. Implement Mean Squared Error and Mean Absolute Error Loss functions, both vec
y_true = np.array([1.0, 2.0, 3.0])
y_pred = np.array([1.1, 1.9, 3.2])

mse = np.mean((y_true - y_pred) ** 2)
mae = np.mean(np.abs(y_true - y_pred))

print(f"MSE: {mse:.4f}, MAE: {mae:.4f}")
```

MSE: 0.0200, MAE: 0.1333

```
In [216... # 50. Vectorized PSNR computation over image batches (Batch, H, W, C) without Pytho
def batch_psnr(img_batch1, img_batch2, max_val=255.0):
    # Compute MSE across spatial and color channels dimensions (axis=1,2,3)
    mse = np.mean((img_batch1 - img_batch2) ** 2, axis=(1, 2, 3))

    # Handle absolute clean-match cases to prevent division by zero errors
    mse = np.where(mse == 0, 1e-10, mse)

    psnr = 20 * np.log10(max_val) - 10 * np.log10(mse)
    return psnr

# Mock batches: 2 images, 32x32 resolution, 3 RGB channels
b1 = np.random.randint(0, 256, (2, 32, 32, 3)).astype(float)
b2 = b1 + np.random.normal(0, 2, (2, 32, 32, 3)) # Add minor noise

print("PSNR per image in batch:", batch_psnr(b1, b2))
```

PSNR per image in batch: [41.96438438 42.06815341]

In []:

Dataset Reading 23

1.World Cup Matches

```
In [217... #using file name
df = pd.read_csv('WorldCupMatches.csv')
df
```

Out[217...

	Year	Datetime	Stage	Stadium	City	Home Team Name	Home Team Goals	Away Team Goals	Away Team Name
0	1930.0	13 Jul 1930 - 15:00	Group 1	Pocitos	Montevideo	France	4.0	1.0	Mexico
1	1930.0	13 Jul 1930 - 15:00	Group 4	Parque Central	Montevideo	USA	3.0	0.0	Belgium
2	1930.0	14 Jul 1930 - 12:45	Group 2	Parque Central	Montevideo	Yugoslavia	2.0	1.0	Brazil
3	1930.0	14 Jul 1930 - 14:50	Group 3	Pocitos	Montevideo	Romania	3.0	1.0	Peru
4	1930.0	15 Jul 1930 - 16:00	Group 1	Parque Central	Montevideo	Argentina	1.0	0.0	France
...	...	...	...	...	...	...	...	...	...
4567	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4568	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4569	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4570	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4571	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

4572 rows × 20 columns



In [218...

```
#using file path
df = pd.read_csv("D:/machine learning/WorldCupMatches.csv")
df
```

Out[218...

	Year	Datetime	Stage	Stadium	City	Home Team Name	Home Team Goals	Away Team Goals	Away Team Name
0	1930.0	13 Jul 1930 - 15:00	Group 1	Pocitos	Montevideo	France	4.0	1.0	Mexico
1	1930.0	13 Jul 1930 - 15:00	Group 4	Parque Central	Montevideo	USA	3.0	0.0	Belgium
2	1930.0	14 Jul 1930 - 12:45	Group 2	Parque Central	Montevideo	Yugoslavia	2.0	1.0	Brazil
3	1930.0	14 Jul 1930 - 14:50	Group 3	Pocitos	Montevideo	Romania	3.0	1.0	Peru
4	1930.0	15 Jul 1930 - 16:00	Group 1	Parque Central	Montevideo	Argentina	1.0	0.0	France
...	...	...	...	...	...	...	...	...	...
4567	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4568	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4569	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4570	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4571	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

4572 rows × 20 columns



In []:

2 liver_synthetic

In [219...

```
#using file name
df = pd.read_csv('liver_synthetic.csv')
df
```

Out[219...

	Age	Gender	TB	DB	Alkphos	Sgpt	Sgot	TP	ALB	AG	LiverPatient
0	55	Male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	Male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	Male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	Male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	Male	3.2	1.54	348	270	339	6.7	2.0	1.05	0
...	...	...	...	...	...	...	...	...	...	...	...
198	47	Male	6.9	1.31	573	68	318	5.8	2.7	1.37	1
199	35	Male	0.2	1.18	582	81	17	8.8	3.4	0.68	1
200	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
201	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
202	36	Male	2.1	1.20	177	218	29	4.9	2.3	0.76	1

203 rows × 11 columns

In [220...

```
#using file path
df = pd.read_csv("D:/machine learning/liver_synthetic.csv")
df
```

Out[220...

	Age	Gender	TB	DB	Alkphos	Sgpt	Sgot	TP	ALB	AG	LiverPatient
0	55	Male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	Male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	Male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	Male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	Male	3.2	1.54	348	270	339	6.7	2.0	1.05	0
...	...	...	...	...	...	...	...	...	...	...	...
198	47	Male	6.9	1.31	573	68	318	5.8	2.7	1.37	1
199	35	Male	0.2	1.18	582	81	17	8.8	3.4	0.68	1
200	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
201	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
202	36	Male	2.1	1.20	177	218	29	4.9	2.3	0.76	1

203 rows × 11 columns

In []:

3 Advertising

```
In [221... #using file name  
df = pd.read_csv('Advertising.csv')  
df
```

```
Out[221...      TV  radio  newspaper  sales  
0  230.1   37.8         69.2   22.1  
1   44.5   39.3         45.1   10.4  
2   17.2   45.9         69.3    9.3  
3  151.5   41.3         58.5   18.5  
4  180.8   10.8         58.4   12.9  
...    ...    ...         ...    ...  
195   38.2    3.7         13.8    7.6  
196   94.2    4.9          8.1    9.7  
197  177.0    9.3          6.4   12.8  
198  283.6   42.0         66.2   25.5  
199  232.1    8.6          8.7   13.4
```

200 rows × 4 columns

```
In [222... #using file path  
df = pd.read_csv("D:/machine learning/Advertising.csv")  
df
```

Out[222...]

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9
...	...	...	...	...
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	9.7
197	177.0	9.3	6.4	12.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	13.4

200 rows × 4 columns

In []:

4 Boston

In [223...]

```
#using file name  
df = pd.read_csv('Boston.csv')  
df
```

Out[223...

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandC
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	
...	...	...	...	...	...	...	...	...	
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	

1460 rows × 81 columns



In [224...

```
#using file path
df = pd.read_csv("D:/machine learning/Boston.csv")
df
```

Out[224...

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandC
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	
...	...	...	...	...	...	...	...	...	
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	

1460 rows × 81 columns



In []:

5 claimants sample

```
In [225... #using file name
df = pd.read_csv('claimants sample.csv')
df
```

Out[225...

	CASENUM	CLMSEX	CLMINSUR	SEATBELT	CLMAGE	LOSS	ATTORNEY
0	5	0	1.0	0	50.0	34.940	0
1	3	1	0.0	0	18.0	0.891	1
2	66	0	1.0	0	5.0	0.330	1
3	70	1	1.0	1	31.0	0.037	0
4	96	0	1.0	0	30.0	NaN	1
5	97	1	1.0	0	35.0	0.309	0
6	10	0	NaN	0	9.0	3.538	0
7	36	1	NaN	0	34.0	4.881	0
8	51	1	1.0	0	60.0	0.874	1
9	55	1	1.0	0	NaN	0.350	1

```
In [226... #using file path
df = pd.read_csv("D:/machine learning/claimants sample.csv")
df
```

Out[226...

	CASENUM	CLMSEX	CLMINSUR	SEATBELT	CLMAGE	LOSS	ATTORNEY
0	5	0	1.0	0	50.0	34.940	0
1	3	1	0.0	0	18.0	0.891	1
2	66	0	1.0	0	5.0	0.330	1
3	70	1	1.0	1	31.0	0.037	0
4	96	0	1.0	0	30.0	NaN	1
5	97	1	1.0	0	35.0	0.309	0
6	10	0	NaN	0	9.0	3.538	0
7	36	1	NaN	0	34.0	4.881	0
8	51	1	1.0	0	60.0	0.874	1
9	55	1	1.0	0	NaN	0.350	1

In []:

6 claimants

```
In [227... #using file name
df = pd.read_csv('claimants.csv')
df
```

Out[227...

	CASENUM	CLMSEX	CLMINSUR	SEATBELT	CLMAGE	LOSS	ATTORNEY
0	5	0.0	1.0	0.0	50.0	34.940	0
1	3	1.0	0.0	0.0	18.0	0.891	1
2	66	0.0	1.0	0.0	5.0	0.330	1
3	70	0.0	1.0	1.0	31.0	0.037	0
4	96	0.0	1.0	0.0	30.0	0.038	1
...	...	...	...	...	...	...	...
1335	34100	0.0	1.0	0.0	NaN	0.576	1
1336	34110	1.0	1.0	0.0	46.0	3.705	0
1337	34113	1.0	1.0	0.0	39.0	0.099	1
1338	34145	1.0	0.0	0.0	8.0	3.177	0
1339	34153	1.0	1.0	0.0	30.0	0.688	1

1340 rows × 7 columns

```
In [228... #using file path
df = pd.read_csv("D:/machine learning/claimants.csv")
df
```

Out[228...

	CASENUM	CLMSEX	CLMINSUR	SEATBELT	CLMAGE	LOSS	ATTORNEY
0	5	0.0	1.0	0.0	50.0	34.940	0
1	3	1.0	0.0	0.0	18.0	0.891	1
2	66	0.0	1.0	0.0	5.0	0.330	1
3	70	0.0	1.0	1.0	31.0	0.037	0
4	96	0.0	1.0	0.0	30.0	0.038	1
...	...	...	...	...	...	...	...
1335	34100	0.0	1.0	0.0	NaN	0.576	1
1336	34110	1.0	1.0	0.0	46.0	3.705	0
1337	34113	1.0	1.0	0.0	39.0	0.099	1
1338	34145	1.0	0.0	0.0	8.0	3.177	0
1339	34153	1.0	1.0	0.0	30.0	0.688	1

1340 rows × 7 columns

In []:

7 gene_expression

In [229...

```
#using file name
df = pd.read_csv('gene_expression.csv')
df
```

Out[229...

	Gene One	Gene Two	Cancer Present
0	4.3	3.9	1
1	2.5	6.3	0
2	5.7	3.9	1
3	6.1	6.2	0
4	7.4	3.4	1
...	...	...	...
2995	5.0	6.5	1
2996	3.4	6.6	0
2997	2.7	6.5	0
2998	3.3	5.6	0
2999	4.6	8.2	0

3000 rows × 3 columns

In [230...

```
#using file path
df = pd.read_csv("D:/machine learning/gene_expression.csv")
df
```

Out[230...

	Gene One	Gene Two	Cancer Present
0	4.3	3.9	1
1	2.5	6.3	0
2	5.7	3.9	1
3	6.1	6.2	0
4	7.4	3.4	1
...	...	...	...
2995	5.0	6.5	1
2996	3.4	6.6	0
2997	2.7	6.5	0
2998	3.3	5.6	0
2999	4.6	8.2	0

3000 rows × 3 columns

In []:

8 hearing_test

```
In [231... #using file name  
df = pd.read_csv('hearing_test.csv')  
df
```

```
Out[231...      age  physical_score  test_result  
0    33.0             40.7            1  
1    50.0             37.2            1  
2    52.0             24.7            0  
3    56.0             31.0            0  
4    35.0             42.9            1  
...     ...             ...           ...  
4995  73.0              3.9            0  
4996  57.0             33.9            1  
4997  49.0             34.5            1  
4998  38.0             46.4            1  
4999  48.0             38.5            1
```

5000 rows × 3 columns

```
In [232... #using file path  
df = pd.read_csv("D:/machine learning/hearing_test.csv")  
df
```


Out[232...]

	age	physical_score	test_result
0	33.0	40.7	1
1	50.0	37.2	1
2	52.0	24.7	0
3	56.0	31.0	0
4	35.0	42.9	1
...	...	...	...
4995	73.0	3.9	0
4996	57.0	33.9	1
4997	49.0	34.5	1
4998	38.0	46.4	1
4999	48.0	38.5	1

5000 rows × 3 columns

In []:

9 homeprices

In [233...]

```
#using file name  
df = pd.read_csv('homeprices.csv')  
df
```

Out[233...

	town	area	price
0	Chennai	2600	5500000
1	Chennai	3000	5650000
2	Chennai	3200	6100000
3	Chennai	3600	6800000
4	Bangalore	2600	5850000
5	Bangalore	2800	6150000
6	Bangalore	3300	6500000
7	Bangalore	3600	7100000
8	Hyderabad	2600	5750000
9	Hyderabad	2900	6000000
10	Hyderabad	3100	6200000
11	Hyderabad	3600	6950000

In [234...

```
#using file path
df = pd.read_csv("D:/machine learning/homeprices.csv")
df
```

Out[234...

	town	area	price
0	Chennai	2600	5500000
1	Chennai	3000	5650000
2	Chennai	3200	6100000
3	Chennai	3600	6800000
4	Bangalore	2600	5850000
5	Bangalore	2800	6150000
6	Bangalore	3300	6500000
7	Bangalore	3600	7100000
8	Hyderabad	2600	5750000
9	Hyderabad	2900	6000000
10	Hyderabad	3100	6200000
11	Hyderabad	3600	6950000

In []:

10 liver_synthetic

```
In [235... #using file name
df = pd.read_csv('liver_synthetic.csv')
df
```

```
Out[235...
```

	Age	Gender	TB	DB	Alkphos	Sgpt	Sgot	TP	ALB	AG	LiverPatient
0	55	Male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	Male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	Male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	Male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	Male	3.2	1.54	348	270	339	6.7	2.0	1.05	0
...	...	...	...	...	...	...	...	...	...	...	...
198	47	Male	6.9	1.31	573	68	318	5.8	2.7	1.37	1
199	35	Male	0.2	1.18	582	81	17	8.8	3.4	0.68	1
200	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
201	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
202	36	Male	2.1	1.20	177	218	29	4.9	2.3	0.76	1

203 rows × 11 columns

```
In [236... #using file path
df = pd.read_csv("D:/machine learning/liver_synthetic.csv")
df
```

Out[236...

	Age	Gender	TB	DB	Alkphos	Sgpt	Sgot	TP	ALB	AG	LiverPatient
0	55	Male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	Male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	Male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	Male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	Male	3.2	1.54	348	270	339	6.7	2.0	1.05	0
...	...	...	...	...	...	...	...	...	...	...	...
198	47	Male	6.9	1.31	573	68	318	5.8	2.7	1.37	1
199	35	Male	0.2	1.18	582	81	17	8.8	3.4	0.68	1
200	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
201	86	Female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
202	36	Male	2.1	1.20	177	218	29	4.9	2.3	0.76	1

203 rows × 11 columns

In []:

11 liver

In [237...

```
#using file name
df = pd.read_csv('liver.csv')
df
```

Out[237...

	Age	Gender	TB	DB	Alkphos	Sgpt	Sgot	TP	ALB	AG	LiverPatient
0	65	Female	0.7	0.1	187	16	18	6.8	3.3	0.90	1
1	62	Male	10.9	5.5	699	64	100	7.5	3.2	0.74	1
2	62	Male	7.3	4.1	490	60	68	7.0	3.3	0.89	1
3	58	Male	1.0	0.4	182	14	20	6.8	3.4	1.00	1
4	72	Male	3.9	2.0	195	27	59	7.3	2.4	0.40	1
...	...	...	...	...	...	...	...	...	...	...	...
578	60	Male	0.5	0.1	500	20	34	5.9	1.6	0.37	0
579	40	Male	0.6	0.1	98	35	31	6.0	3.2	1.10	1
580	52	Male	0.8	0.2	245	48	49	6.4	3.2	1.00	1
581	31	Male	1.3	0.5	184	29	32	6.8	3.4	1.00	1
582	38	Male	1.0	0.3	216	21	24	7.3	4.4	1.50	0

583 rows × 11 columns

In [238...

```
#using file path
df = pd.read_csv("D:/machine learning/liver.csv")
df
```

Out[238...

	Age	Gender	TB	DB	Alkphos	Sgpt	Sgot	TP	ALB	AG	LiverPatient
0	65	Female	0.7	0.1	187	16	18	6.8	3.3	0.90	1
1	62	Male	10.9	5.5	699	64	100	7.5	3.2	0.74	1
2	62	Male	7.3	4.1	490	60	68	7.0	3.3	0.89	1
3	58	Male	1.0	0.4	182	14	20	6.8	3.4	1.00	1
4	72	Male	3.9	2.0	195	27	59	7.3	2.4	0.40	1
...	...	...	...	...	...	...	...	...	...	...	...
578	60	Male	0.5	0.1	500	20	34	5.9	1.6	0.37	0
579	40	Male	0.6	0.1	98	35	31	6.0	3.2	1.10	1
580	52	Male	0.8	0.2	245	48	49	6.4	3.2	1.00	1
581	31	Male	1.3	0.5	184	29	32	6.8	3.4	1.00	1
582	38	Male	1.0	0.3	216	21	24	7.3	4.4	1.50	0

583 rows × 11 columns

In []:

12 mouse_viral_study

```
In [239... #using file name  
df = pd.read_csv('mouse_viral_study.csv')  
df
```

```
Out[239...      Med_1_mL  Med_2_mL  Virus Present  
0      6.508231    8.582531            0  
1      4.126116    3.073459            1  
2      6.427870    6.369758            0  
3      3.672953    4.905215            1  
4      1.580321    2.440562            1  
...      ...      ...      ...  
395    2.884122    3.271748            1  
396    7.290855    9.488672            0  
397    7.895325    8.272529            0  
398    2.690592    2.674979            1  
399    7.587821    9.473968            0
```

400 rows × 3 columns

```
In [240... #using file path  
df = pd.read_csv("D:/machine learning/mouse_viral_study.csv")  
df
```

Out[240...

	Med_1_mL	Med_2_mL	Virus Present
0	6.508231	8.582531	0
1	4.126116	3.073459	1
2	6.427870	6.369758	0
3	3.672953	4.905215	1
4	1.580321	2.440562	1
...	...	...	...
395	2.884122	3.271748	1
396	7.290855	9.488672	0
397	7.895325	8.272529	0
398	2.690592	2.674979	1
399	7.587821	9.473968	0

400 rows × 3 columns

In []:

13 mushrooms

In [241...

```
#using file name
df = pd.read_csv('mushrooms.csv')
df
```

Out[241...

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color	...	su t
0	p	x	s	n	t	p	f	c	n	k	...	
1	e	x	s	y	t	a	f	c	b	k	...	
2	e	b	s	w	t	l	f	c	b	n	...	
3	p	x	y	w	t	p	f	c	n	n	...	
4	e	x	s	g	f	n	f	w	b	k	...	
...	...	...	...	...	...	...	...	...	...	...	...	
8119	e	k	s	n	f	n	a	c	b	y	...	
8120	e	x	s	n	f	n	a	c	b	y	...	
8121	e	f	s	n	f	n	a	c	b	n	...	
8122	p	k	y	n	f	y	f	c	n	b	...	
8123	e	x	s	n	f	n	a	c	b	y	...	

8124 rows × 23 columns



In [242...

```
#using file path
df = pd.read_csv("D:/machine learning/mushrooms.csv")
df
```


Out[242...

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color	...	su t
0	p	x	s	n	t	p	f	c	n	k	...	
1	e	x	s	y	t	a	f	c	b	k	...	
2	e	b	s	w	t	l	f	c	b	n	...	
3	p	x	y	w	t	p	f	c	n	n	...	
4	e	x	s	g	f	n	f	w	b	k	...	
...	...	...	...	...	...	...	...	...	...	...	...	
8119	e	k	s	n	f	n	a	c	b	y	...	
8120	e	x	s	n	f	n	a	c	b	y	...	
8121	e	f	s	n	f	n	a	c	b	n	...	
8122	p	k	y	n	f	y	f	c	n	b	...	
8123	e	x	s	n	f	n	a	c	b	y	...	

8124 rows × 23 columns



In []:

14 penguins_size

In [243...

```
#using file name
df = pd.read_csv('penguins_size.csv')
df
```

Out[243...

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_n
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	
...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	
340	Gentoo	Biscoe	46.8	14.3	215.0	
341	Gentoo	Biscoe	50.4	15.7	222.0	
342	Gentoo	Biscoe	45.2	14.8	212.0	
343	Gentoo	Biscoe	49.9	16.1	213.0	

344 rows × 7 columns

In [244...

#using file path
df = pd.read_csv("D:/machine learning/penguins_size.csv")
df

Out[244...

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_n
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	
...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	
340	Gentoo	Biscoe	46.8	14.3	215.0	
341	Gentoo	Biscoe	50.4	15.7	222.0	
342	Gentoo	Biscoe	45.2	14.8	212.0	
343	Gentoo	Biscoe	49.9	16.1	213.0	

344 rows × 7 columns

In []:

15 stroke prediction

```
In [245... #using file name
df = pd.read_csv('stroke prediction.csv')
df
```

Out[245...

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Resi
0	30669	Male	3.0	0	0	No	children	
1	30468	Male	58.0	1	0	Yes	Private	
2	16523	Female	8.0	0	0	No	Private	
3	56543	Female	70.0	0	0	Yes	Private	
4	46136	Male	14.0	0	0	No	Never_worked	
...	...	...	...	...	...	...	...	...
43395	56196	Female	10.0	0	0	No	children	
43396	5450	Female	56.0	0	0	Yes	Govt_job	
43397	28375	Female	82.0	1	0	Yes	Private	
43398	27973	Male	40.0	0	0	Yes	Private	
43399	36271	Female	82.0	0	0	Yes	Private	

43400 rows × 12 columns

```
In [246... #using file path
df = pd.read_csv("D:/machine learning/stroke prediction.csv")
df
```

Out[246...

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Resi
0	30669	Male	3.0	0	0	No	children	
1	30468	Male	58.0	1	0	Yes	Private	
2	16523	Female	8.0	0	0	No	Private	
3	56543	Female	70.0	0	0	Yes	Private	
4	46136	Male	14.0	0	0	No	Never_worked	
...	...	...	...	...	...	...	...	...
43395	56196	Female	10.0	0	0	No	children	
43396	5450	Female	56.0	0	0	Yes	Govt_job	
43397	28375	Female	82.0	1	0	Yes	Private	
43398	27973	Male	40.0	0	0	Yes	Private	
43399	36271	Female	82.0	0	0	Yes	Private	

43400 rows × 12 columns



In []:

16 tips

In [247...

```
#using file name
df = pd.read_csv('tips.csv')
df
```

Out[247...

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

In [248...

```
#using file path
df = pd.read_csv("D:/machine learning/tips.csv")
df
```

Out[248...

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

In []:

17 titanic

In [249...

```
#using file name  
df = pd.read_csv('titanic.csv')  
df
```

Out[249...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05
...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.73

891 rows × 12 columns



In [250...

```
#using file path  
df = pd.read_csv("D:/machine learning/titanic.csv")  
df
```


Out[250...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05
...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.73

891 rows × 12 columns



In []:

18 train_test

In [251...

```
#using file name  
df = pd.read_csv('train_test.csv')  
df
```

Out[251...

	advitise	sales
0	2	15
1	3	17
2	7	12
3	9	8
4	4	3
5	5	4
6	11	6
7	13	8
8	10	13
9	6	17
10	15	4
11	19	3
12	2	8
13	3	1
14	6	14
15	9	17
16	2	14
17	8	16
18	1	11
19	0	12

In [252...

```
#using file path  
df = pd.read_csv("D:/machine learning/train_test.csv")  
df
```

Out[252...

	advitise	sales
0	2	15
1	3	17
2	7	12
3	9	8
4	4	3
5	5	4
6	11	6
7	13	8
8	10	13
9	6	17
10	15	4
11	19	3
12	2	8
13	3	1
14	6	14
15	9	17
16	2	14
17	8	16
18	1	11
19	0	12

In []:

19 Bahaman

In [253...

```
#using file name  
df = pd.read_excel('Bahaman.xlsx')  
df
```

Out[253...

	Defective	Country
0	0	India
1	0	India
2	0	India
3	0	India
4	1	India
...	...	...
795	0	Bangladesh
796	0	Bangladesh
797	1	Bangladesh
798	0	Bangladesh
799	0	Bangladesh

800 rows × 2 columns

In [254...

```
#using file path
df = pd.read_excel("D:/machine learning/Bahaman.xlsx")
df
```

Out[254...

	Defective	Country
0	0	India
1	0	India
2	0	India
3	0	India
4	1	India
...	...	...
795	0	Bangladesh
796	0	Bangladesh
797	1	Bangladesh
798	0	Bangladesh
799	0	Bangladesh

800 rows × 2 columns

In []:

20 Bolt diameter

```
In [255... #using file name  
df = pd.read_excel('Bolt diameter.xlsx')  
df
```

Out[255...

	Diameter
0	11.27
1	12.06
2	12.15
3	9.89
4	10.82
5	13.53
6	10.83
7	8.87
8	11.14
9	9.71
10	11.88
11	9.53
12	13.17
13	10.54
14	11.25
15	10.84
16	11.29
17	10.54
18	10.74
19	10.93

```
In [256... #using file path  
df = pd.read_excel("D:/machine learning/Bolt diameter.xlsx")  
df
```

Out[256...

	Diameter
0	11.27
1	12.06
2	12.15
3	9.89
4	10.82
5	13.53
6	10.83
7	8.87
8	11.14
9	9.71
10	11.88
11	9.53
12	13.17
13	10.54
14	11.25
15	10.84
16	11.29
17	10.54
18	10.74
19	10.93

In []:

21 Contract Renewal

In [257...

```
#using file name  
df = pd.read_excel('Contract Renewal.xlsx')  
df
```

Out[257...

	SupplierA	SupplierB	SupplierC
0	6.15	7.87	7.41
1	6.22	5.21	3.61
2	6.76	7.94	7.23
3	4.29	7.36	5.53
4	7.08	6.17	3.97
...	...	...	...
115	5.85	4.53	4.27
116	5.04	8.06	6.39
117	5.68	7.14	4.60
118	4.77	4.84	6.15
119	3.57	4.55	5.28

120 rows × 3 columns

In [258...

```
#using file path
df = pd.read_excel("D:/machine learning/Contract Renewal.xlsx")
df
```

Out[258...

	SupplierA	SupplierB	SupplierC
0	6.15	7.87	7.41
1	6.22	5.21	3.61
2	6.76	7.94	7.23
3	4.29	7.36	5.53
4	7.08	6.17	3.97
...	...	...	...
115	5.85	4.53	4.27
116	5.04	8.06	6.39
117	5.68	7.14	4.60
118	4.77	4.84	6.15
119	3.57	4.55	5.28

120 rows × 3 columns

In []:

22 Fabric data

```
In [259... #using file name  
df = pd.read_excel('Fabric data.xlsx')  
df
```

```
Out[259... Fabric_length
```

0	151.2
1	160.3
2	147.5
3	149.2
4	159.2
5	155.6
6	148.1
7	154.8
8	156.8
9	167.5
10	164.5
11	147.9
12	154.1
13	159.4
14	147.5
15	148.2
16	154.2
17	148.1
18	163.1
19	155.2
20	154.6
21	155.4
22	158.2
23	157.6
24	158.4

In [260...

```
#using file path
df = pd.read_excel("D:/machine learning/Fabric data.xlsx")
df
```

Out[260...

	Fabric_length
0	151.2
1	160.3
2	147.5
3	149.2
4	159.2
5	155.6
6	148.1
7	154.8
8	156.8
9	167.5
10	164.5
11	147.9
12	154.1
13	159.4
14	147.5
15	148.2
16	154.2
17	148.1
18	163.1
19	155.2
20	154.6
21	155.4
22	158.2
23	157.6
24	158.4

In []:

```
In [261... #using file name
df = pd.read_excel('JohnyTalkers.xlsx')
df
```

```
Out[261...
      Person  Drinks
0    Adults  Did Not Purchase
1    Adults  Did Not Purchase
2    Adults  Did Not Purchase
3    Adults  Did Not Purchase
4    Adults  Did Not Purchase
...      ...      ...
1215  Children  Did Not Purchase
1216  Children  Did Not Purchase
1217  Children  Did Not Purchase
1218  Children  Did Not Purchase
1219  Children      Purchased
```

1220 rows × 2 columns

```
In [262... #using file path
df = pd.read_excel("D:/machine learning/JohnyTalkers.xlsx")
df
```

Out[262...

	Person	Drinks
0	Adults	Did Not Purchase
1	Adults	Did Not Purchase
2	Adults	Did Not Purchase
3	Adults	Did Not Purchase
4	Adults	Did Not Purchase
...	...	...
1215	Children	Did Not Purchase
1216	Children	Did Not Purchase
1217	Children	Did Not Purchase
1218	Children	Did Not Purchase
1219	Children	Purchased

1220 rows × 2 columns

In []: